CPSY 300 · CLOUD COMPUTING · PROJECT 2

Nutritional Insights Dashboard

Combined Project Summary — Backend, Frontend and Cloud Deployment

Team: Yuandong Yang · Ethan Bayarsaikhan · Justin Norman-Rance | Compiled: 2026-11-06

This document merges the three stage reports. Each part keeps its own verification evidence and screenshots.

Contents

Part	Stage	Contributor
Part 1	Backend — local data processing & Nutritional Insights API	Yuandong Yang
Part 2	Frontend — interactive dashboard & data visualization	Ethan Bayarsaikhan
Part 3	Cloud — Azure Blob Storage, Functions & Static Web Apps	Justin Norman-Rance

Live dashboard: https://salmon-ocean-0ce94150f.4.azurestaticapps.net/ · Live API: https://cpsy300-nutrition-api.azurewebsites.net/api

Part 1

Stage 1 — Backend (Local Data Processing & API)

Contributor: Yuandong Yang

CPSY 300 · CLOUD COMPUTING · PROJECT 2

Stage 1 Backend Summary

Local Data Processing & Nutritional Insights API

Author: Yuandong Yang (Person 1) · Stage 1 | Verified: 2026-11-04

This page summarises the Stage 1 implementation and its evidence. Person numbering follows the team's division of work.

01 / Delivery at a Glance

4	7,806	44	5
GET API endpoints	recipes in the dataset	automated tests passing	diet types

Stage 1 of `Project2/division.md` is delivered as a self-contained folder. It reuses the cleaning, aggregation and test foundation of the existing backend, and adds local CSV loading, text search, a correlation matrix and a k-means implementation written from scratch with NumPy, behind an API contract the front end can build against.

> **Status:** Stage 1 is complete and evidenced. On Python 3.11 all 44 tests pass and flake8 reports nothing; `func start` from Azure Functions Core Tools 4.16.0 registers all four routes; ten real HTTP cases pass, including the 400 / 404 paths and the local CORS response header; four browser screenshots and three terminal screenshots are archived in `docs/screenshots/`. Stage 1 involves no cloud deployment — the group's Azure release, report and video belong to Stages 2 and 3.

02 / Stage 1 Requirements Checklist

Requirement from the division of work	Where it lives and how it is evidenced	Status
Python project, dependencies, host configuration	requirements.txt, host.json, local settings template	Done

Requirement from the division of work	Where it lives and how it is evidenced	Status
Load and clean the local CSV	function_app.py, nutrition.py, data/All_Diets.csv	Verified
Health / Insights / Recipes / Clusters	4 GET routes; every call over the full dataset returns 200	Verified
Endpoint tests, flake8, API documentation	tests/, README.md, docs/verification.txt	Done
Run locally, screenshots, merge to main	func start 4.16.0 registers 4 routes; verify_http.py runs 10 real HTTP cases; 7 images in docs/screenshots/	Verified

Checked against the instructor's phase2-requirements.docx: it asks for cloud deployment, at least three visualizations, interactive controls, execution timing and front-end/back-end integration. It does not specifically require k-means or four APIs — clustering, search and the correlation matrix come from Stage 1 of the team's own division.md.

---

03 / Structure and Data Processing

Local All_Diets.csv → Pandas cleaning → Statistics / NumPy clustering → Azure Functions JSON

Cleaning rules

- Strip whitespace from column names and text; normalise diet and cuisine casing.
- Missing categories become unknown; a missing recipe name becomes Unnamed recipe.
- Invalid, missing, negative and infinite nutrient values are filled with the column's valid mean.
- Empty data, a missing required column, or a column with no valid value at all is rejected.

Project layout

```
stage1/
├── function_app.py
├── nutrition.py / clustering.py
├── host.json / requirements*.txt
├── local.settings.json.example
├── verify_http.py
├── data/All_Diets.csv
├── tests/  (3 test files)
├── docs/   (measured results, verification record,
screenshots)
├── README.md
└── summary.html
```

Stage 1 does not use Blob Storage. The cache is keyed on file path, modification time and size; Stage 3 switches the loader to Azure Blob.

---

04 / API Contract and Front-End Hand-Off

Base URL: `http://localhost:7071/api` . Every endpoint returns JSON, and each data endpoint reports its execution time, source and cache state.

GET route	Input	Main output and purpose
/health	none	Service status; touches no data, so it works as a liveness check.
/insights	none	average_macros → bar chart; summary.diet_counts → pie chart; correlation → heatmap.
/recipes	diet_type, search, page, limit	Recipe rows plus paging fields. search matches recipe name, diet type and cuisine, and combines with the filter.
/clusters	k, default 3, range 1–10	points, centroids, cluster_counts → scatter chart coloured by cluster.

Paging defaults to 50 rows and is capped at 500; pages are 1-based and a page past the end returns the last page. Invalid parameters return 400 and an unknown diet type returns 404. A search with no match returns an empty list. Internal errors return a generic error JSON.

The existing frontend/ still targets the earlier three-endpoint version; the new controls and charts are Person 2's work. The CORS template allows localhost:5500 and 127.0.0.1:5500.

---

05 / Measured Results on the Full Dataset

The figures below come from the cleaned local CSV. All 7,806 recipes take part in the statistics and the clustering.

Recipes per diet type

Diet type	Recipes	Relative size
mediterranean	1,753	
dash	1,745	
vegan	1,522	
keto	1,512	
paleo	1,274	

Average macros (grams)

Diet	Protein	Carbs	Fat
dash	69.28	160.54	101.15
keto	101.27	57.97	153.12
mediterranean	101.11	152.91	101.42

Diet	Protein	Carbs	Fat
paleo	88.67	129.55	135.67
vegan	56.16	254.0	103.3

NumPy k-means

Protein, carbohydrate and fat are the three features. They are z-score standardized first so that no unit scale dominates the distance. The random seed is fixed at 42 and the centroids start from distinct sample vectors; the loop then alternates nearest-centroid assignment and cluster-mean updates for at most 100 iterations. A constant feature uses a scale of 1, and an empty cluster keeps its previous centroid, so no NaN can appear.

Measured grouping at k = 3: cluster 0 holds 1,205 recipes, cluster 1 holds 5,681 and cluster 2 holds 920. The returned points and centroids are converted back to grams, while inertia stays as the sum of squared distances in the standardized space.

> Cluster numbers carry no dietary meaning. This is exploratory grouping, and a single initialisation is not guaranteed to be globally optimal. Undefined values in the correlation matrix are encoded as null.

---

06 / Tests and Evidence

```
$ .venv311/bin/python --version
Python 3.11.18

$ python -m pytest -q
........................................ [100%]
44 passed in 0.49s

$ python -m flake8 . --exclude=.venv,.venv311
Exit code 0 — no findings

$ func start --port 7071
Core Tools Version:       4.16.0
Functions: clusters / health / insights / recipes  [GET] http://localhost:7071/api/...

$ python verify_http.py
PASS 200 GET /api/health
PASS 200 GET /api/insights
PASS 200 GET /api/recipes?diet_type=keto&search=chicken&page=1&limit=5
PASS 200 GET /api/recipes?diet_type=keto&search=chicken&page=2&limit=5
PASS 200 GET /api/recipes?search=stage1-no-match-000000
PASS 200 GET /api/clusters?k=3
PASS 400 GET /api/clusters?k=0
PASS 400 GET /api/clusters?k=11
PASS 400 GET /api/recipes?page=0
PASS 404 GET /api/recipes?diet_type=nonexistent
PASS payload assertions and local CORS; evidence: docs/http-results.json
```

The tests cover cleaning and imputation, the aggregations, endpoint output, filtering and paging, literal search, constant columns in the correlation matrix, k-means reproducibility and grouping, invalid k, local CSV cache refresh, and the error responses.

The actual Azure Functions handlers were also called against the full CSV: all four return status 200, and both the insights total and the clusters point count are 7,806. On top of that, `verify_http.py` sends ten real HTTP requests through the running Functions host and

asserts each one: the success paths, that pages 1 and 2 differ, the empty search result, 400 for an invalid k and an invalid page, 404 for an unknown diet type, and the `Access-Control-Allow-Origin: http://localhost:5500` response header.

Verification record · In-process API results (JSON) · Real HTTP results (JSON) · Run instructions and API reference

> Verification environment: macOS (arm64), Python 3.11.18 (the version the team's project already uses), the isolated `.venv311` virtual environment, Azure Functions Core Tools 4.16.0 and Functions Runtime 4.1054.250.26428. Local host compatibility was confirmed in that environment.

Stage 1 Local-Run Screenshots (2026-11-04)

The seven images below are the actual run evidence on Python 3.11 with Core Tools 4.16.0. Every one was taken with the macOS `screencapture` tool from a real window on screen: the browser images are Google Chrome windows with the address bar visible, running in a separate empty profile so no personal browsing data appears, and the terminal images are real command output in Terminal.app windows. The images are embedded, so this HTML can be sent on its own.

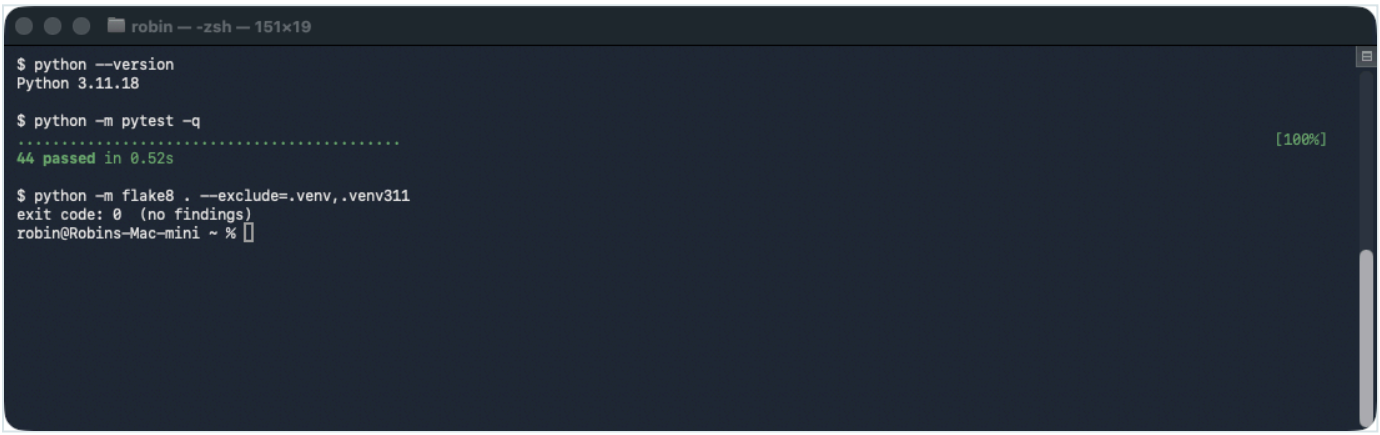


Figure S1 — All 44 pytest tests pass and flake8 exits 0 on Python 3.11. (`docs/screenshots/cli-pytest-flake8.png`)

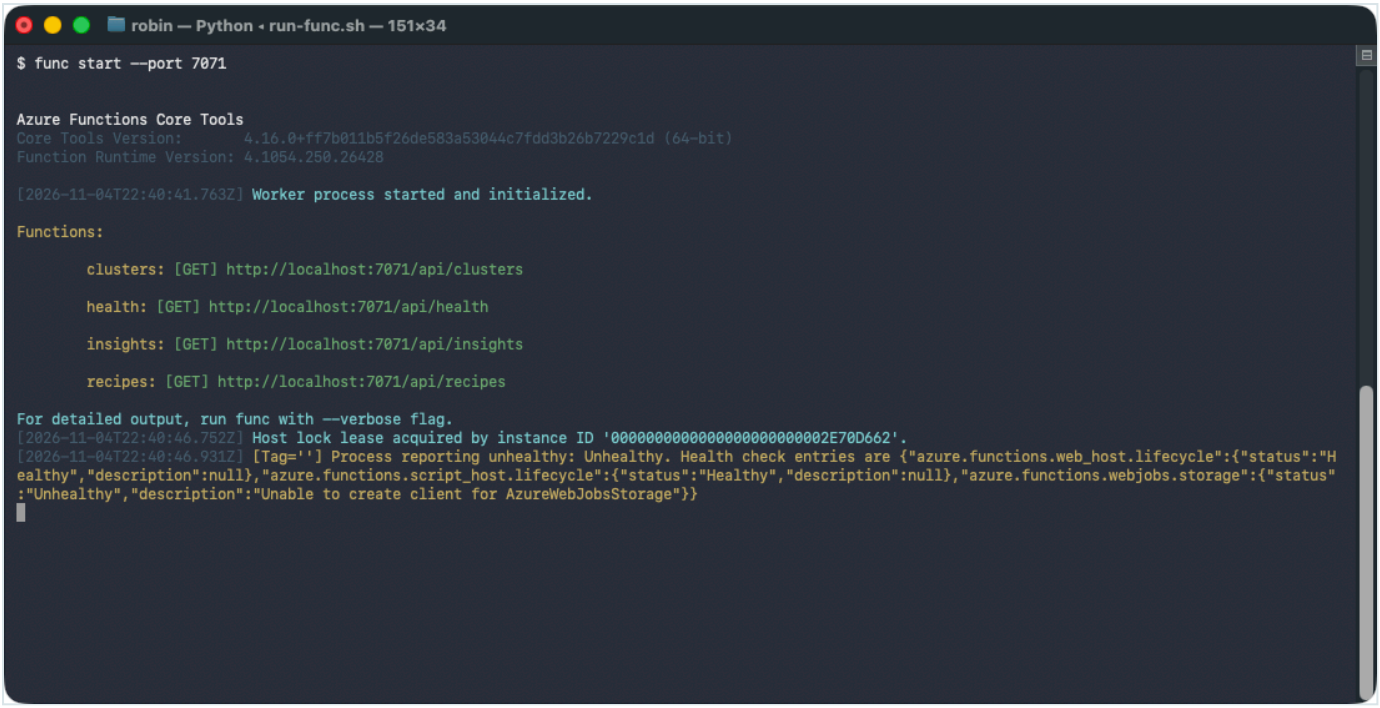


Figure S2 — `func start` brings up the local Functions host; the clusters, health, insights and recipes GET routes are all registered. (`docs/screenshots/cli-func-start.png`)

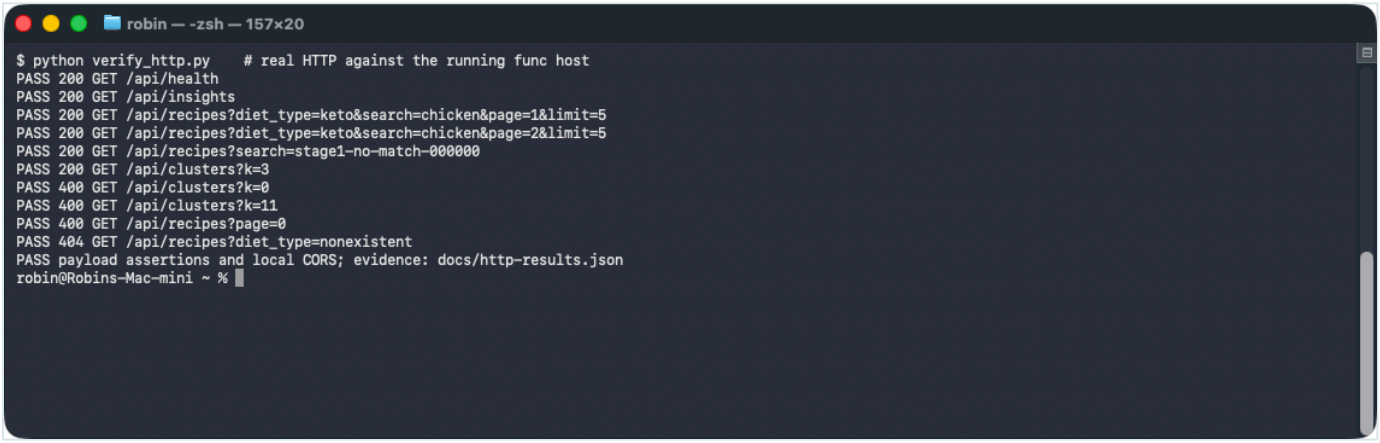


Figure S3 — Ten real HTTP requests through the running host, including the 400 / 404 paths and the local CORS header, all pass. (`docs/screenshots/cli-http-verification.png`)

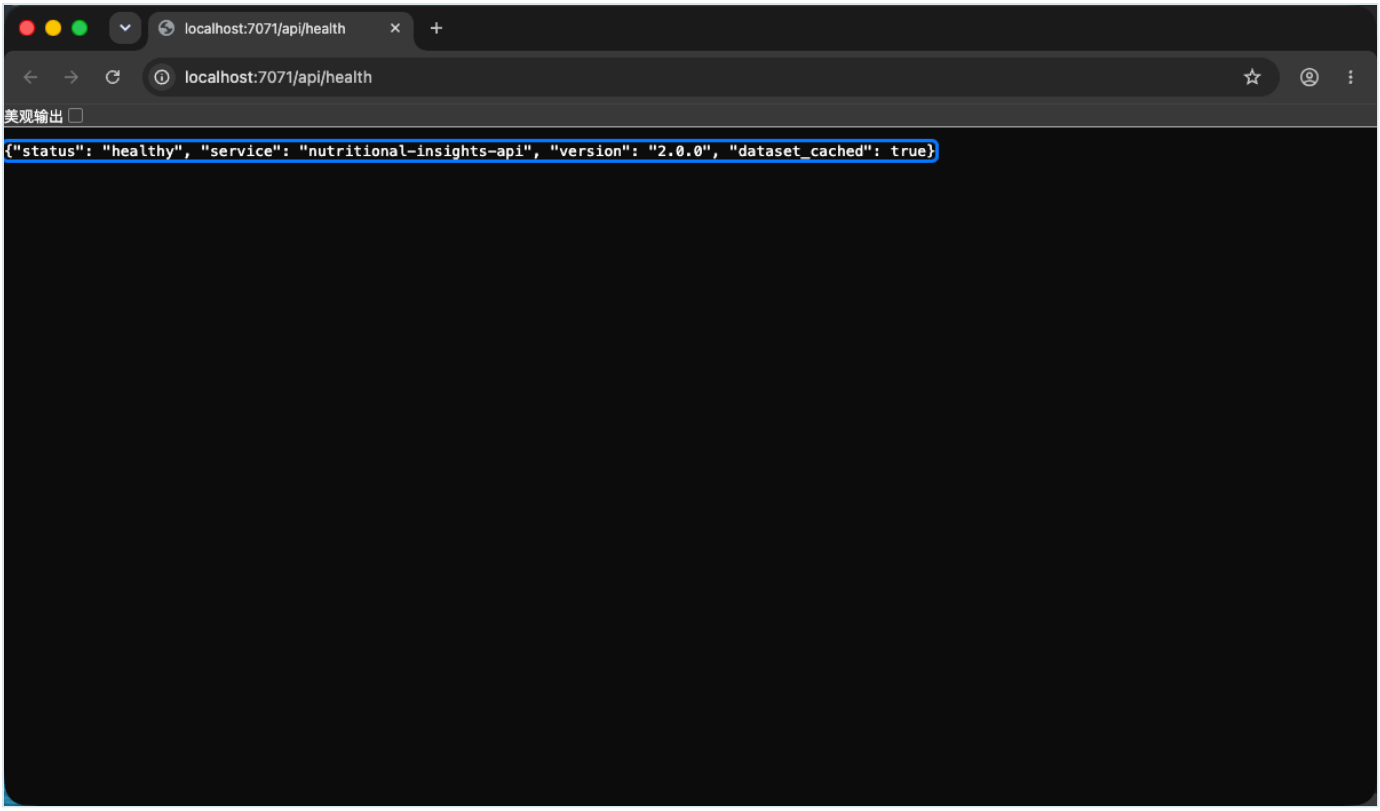


Figure S4 — Browser response from http://localhost:7071/api/health. (`docs/screenshots/api-health-browser.png`)

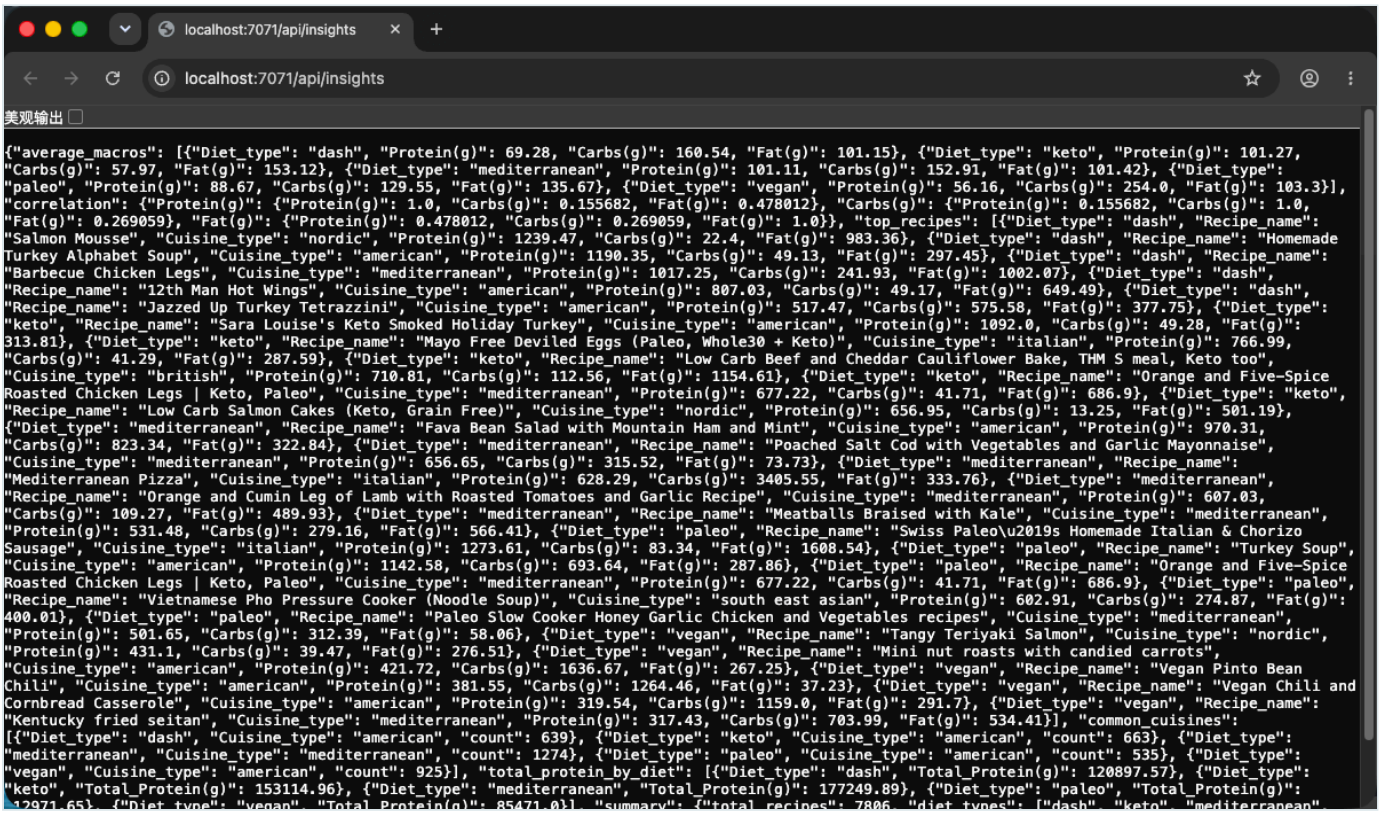


Figure S5 — /api/insights in the browser: average macros per diet, the correlation matrix and the execution time.
(`docs/screenshots/api-insights-browser.png`)

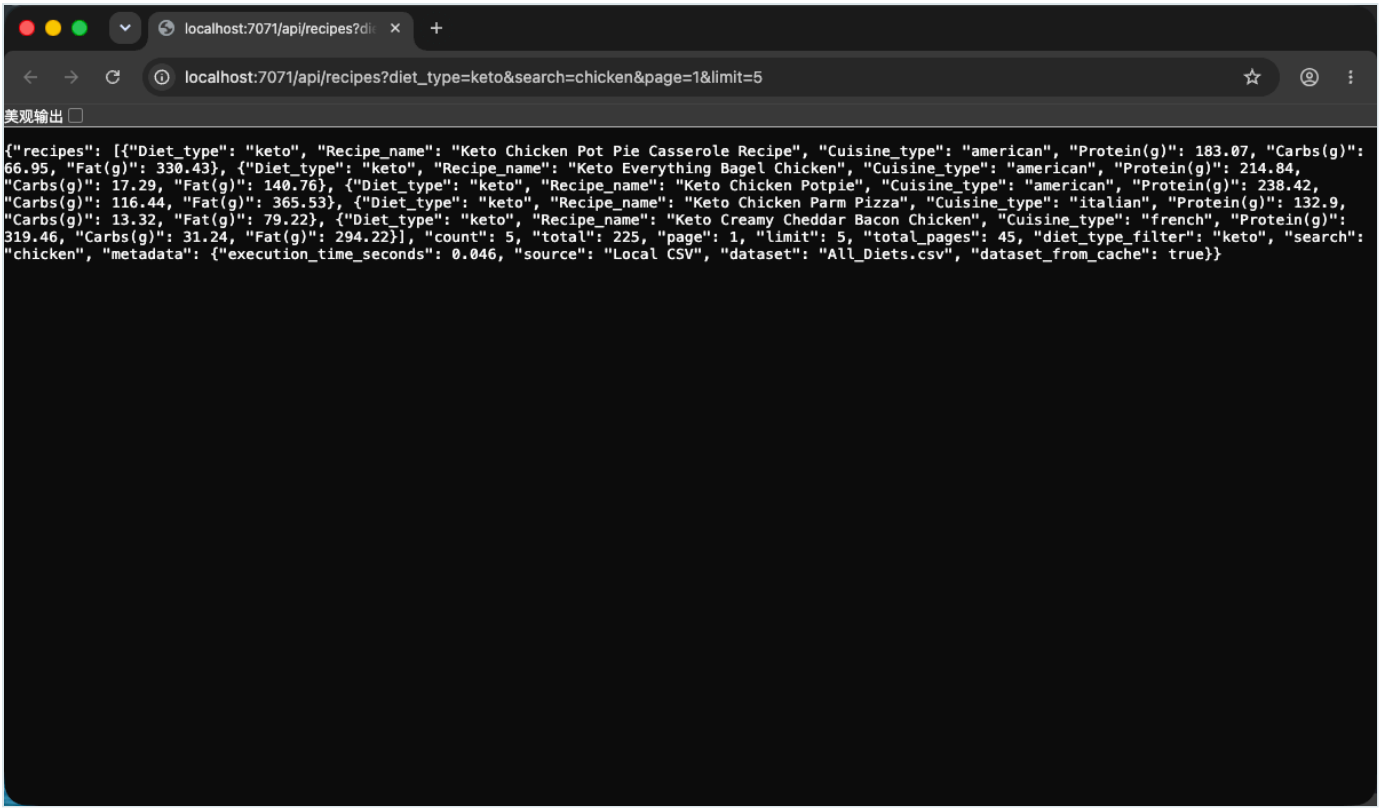


Figure S6 — /api/recipes?diet_type=keto&search=chicken&page=1&limit=5 in the browser: filtering, search and the paging fields. (`docs/screenshots/api-recipes-browser.png`)

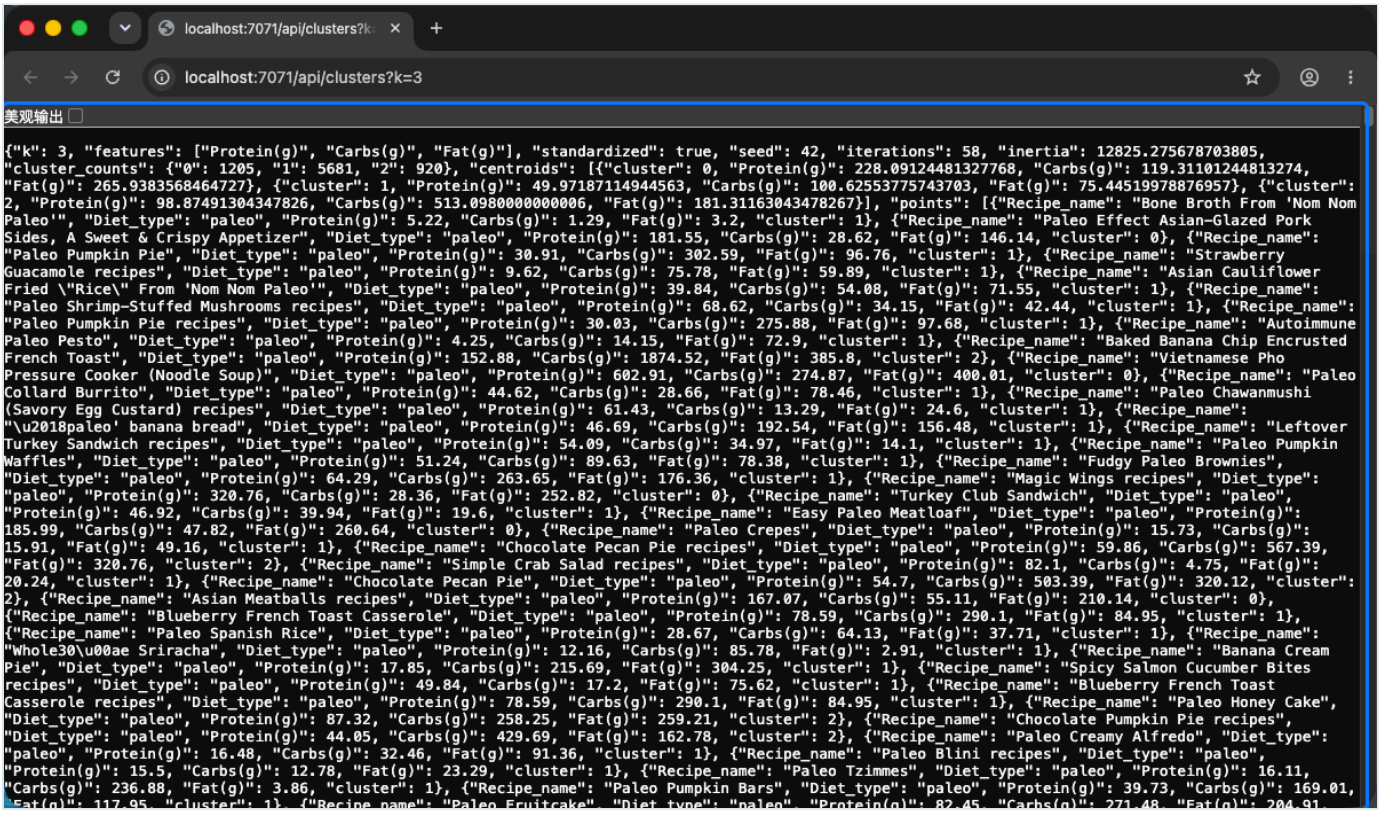


Figure S7 — /api/clusters?k=3 in the browser: cluster sizes, centroids and the scatter points from the hand-written k-means. (`docs/screenshots/api-clusters-browser.png`)

Existing Screenshots and Cloud Evidence

The images below are screenshots the project already held, each one checked against its source. They evidence the deployment and responses of the earlier three-endpoint backend; the new search, correlation and clusters features are evidenced by the Stage 1 record above. The images are embedded, so this HTML can be sent on its own.

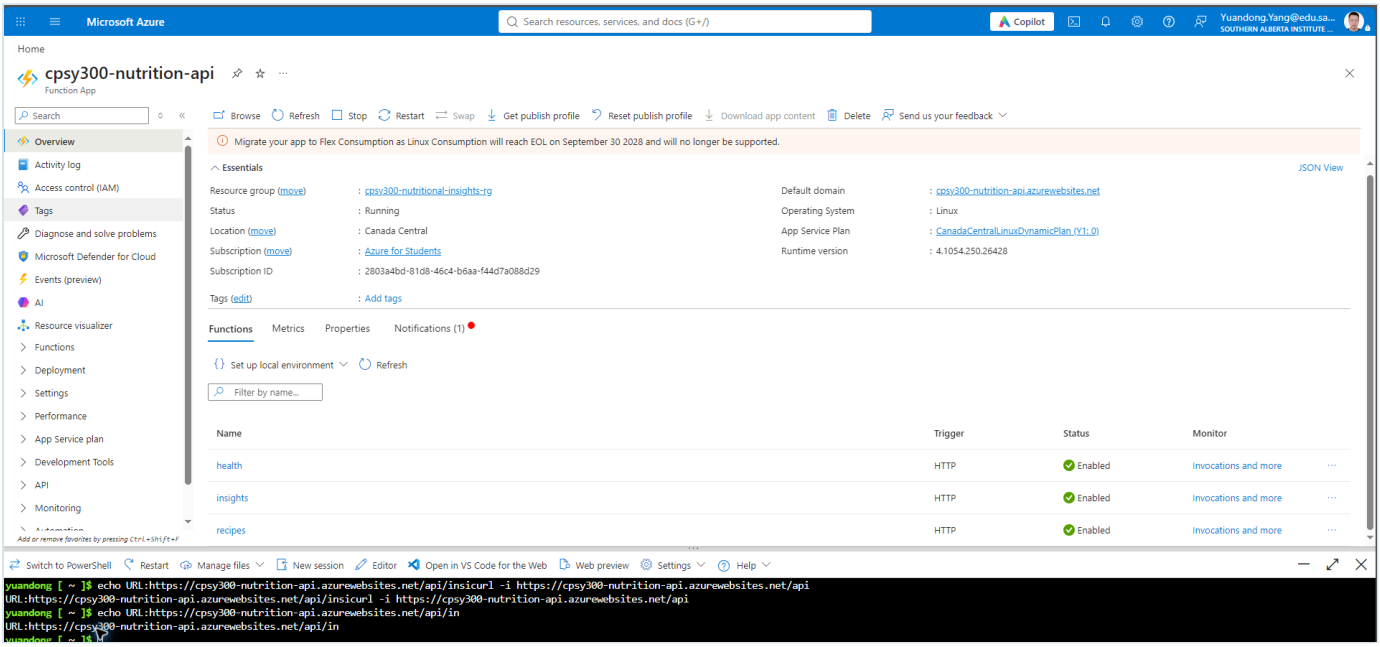


Figure 1 · Existing Azure Function App: Running, with the health, insights and recipes HTTP functions enabled.
Source: Project2/docs/screenshots/portal-function-app.png

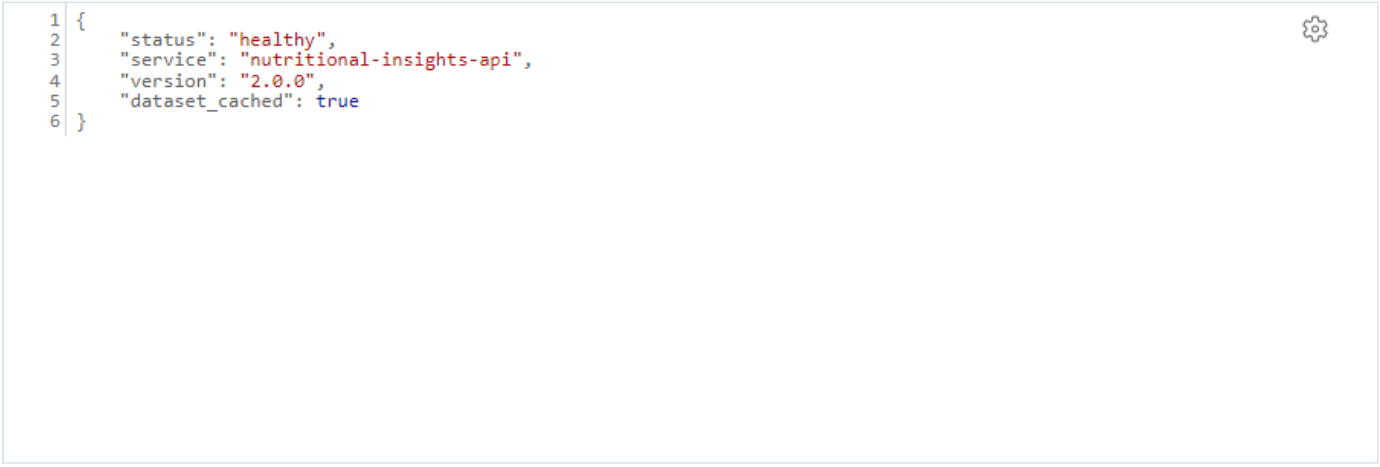


Figure 2 · Existing health response: status=healthy, service version 2.0.0.
Source: Project2/docs/screenshots/api-health-curl.png

```
{
    "average_macros": [
        {
            "Diet_type": "dash",
            "Protein(g)": 69.28,
            "Carbs(g)": 160.54,
            "Fat(g)": 101.15
        },
        {
            "Diet_type": "keto",
            "Protein(g)": 101.27,
            "Carbs(g)": 57.97,
            "Fat(g)": 153.12
        },
        {
            "Diet_type": "mediterranean",
            "Protein(g)": 101.11,
            "Carbs(g)": 152.91,
            "Fat(g)": 101.42
        },
        {
            "Diet_type": "paleo",
            "Protein(g)": 88.67,
            "Carbs(g)": 129.55,
            "Fat(g)": 135.67
        },
        {
            "Diet_type": "vegan",
            "Protein(g)": 56.16,
            "Carbs(g)": 254,
            "Fat(g)": 103.3
        }
    ],
    "top_recipes": [
        {
            "Diet_type": "dash",
            "Recipe_name": "Salmon Mousse",
            "Cuisine_type": "nordic",
            "Protein(g)": 1239.47,
            "Carbs(g)": 22.4,
            "Fat(g)": 983.36
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "Homemade Turkey Alphabet Soup",
            "Cuisine_type": "american",
            "Protein(g)": 1190.35,
            "Carbs(g)": 49.13,
            "Fat(g)": 297.45
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "Barbecue Chicken Legs",
            "Cuisine_type": "mediterranean",
            "Protein(g)": 1017.25,
            "Carbs(g)": 241.93,
            "Fat(g)": 1002.07
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "12th Man Hot Wings",
```


Figure 3 · Existing insights response: average macros for the five diet types and the highest-protein recipes.

Source: Project2/docs/screenshots/api-insights-browser.png

```
{
    "recipes": [
        {
            "Diet_type": "paleo",
            "Recipe_name": "Bone Broth From 'Nom Nom Paleo'",
            "Cuisine_type": "american",
            "Protein(g)": 5.22,
            "Carbs(g)": 1.29,
            "Fat(g)": 3.2
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Paleo Effect Asian-Glazed Pork Sides, A Sweet & Crispy Appetizer",
            "Cuisine_type": "south east asian",
            "Protein(g)": 181.55,
            "Carbs(g)": 28.62,
            "Fat(g)": 146.14
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Paleo Pumpkin Pie",
            "Cuisine_type": "american",
            "Protein(g)": 30.91,
            "Carbs(g)": 302.59,
            "Fat(g)": 96.76
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Strawberry Guacamole recipes",
            "Cuisine_type": "mexican",
            "Protein(g)": 9.62,
            "Carbs(g)": 75.78,
            "Fat(g)": 59.89
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Asian Cauliflower Fried \"Rice\" From 'Nom Nom Paleo'",
            "Cuisine_type": "chinese",
            "Protein(g)": 39.84,
            "Carbs(g)": 54.08,
            "Fat(g)": 71.55
        }
    ],
    "count": 5,
    "total": 7806,
    "page": 1.
```


Figure 4 · Existing recipes response: 5 recipes, a total of 7,806 and the first page of results.
Source: Project2/docs/screenshots/api-recipes-response.png



Figure 5 · Existing CORS and timing record: insights HTTP 200 on 2026-10-20, the measured duration and the allowed front-end origin.
Source: Project2/docs/screenshots/cli-cors-and-timing.png

07 / Sample JSON Responses

Click to expand. These are measured values; insights omits the extra statistics lists and clusters shows only the first two points. The complete payloads are in the JSON files linked above.

▼ GET /api/health · live JSON response


```
{
  "status": "healthy",
  "service": "nutritional-insights-api",
  "version": "2.0.0",
  "dataset_cached": false
}
```

▼ GET /api/insights · live JSON response


```
{
  "average_macros": [
    {
      "Diet_type": "dash",
      "Protein(g)": 69.28,
      "Carbs(g)": 160.54,
      "Fat(g)": 101.15
    },
    {
      "Diet_type": "keto",
      "Protein(g)": 101.27,
      "Carbs(g)": 57.97,
      "Fat(g)": 153.12
    },
    {
      "Diet_type": "mediterranean",
      "Protein(g)": 101.11,
      "Carbs(g)": 152.91,
      "Fat(g)": 101.42
    },
    {
      "Diet_type": "paleo",
      "Protein(g)": 88.67,
      "Carbs(g)": 129.55,
      "Fat(g)": 135.67
    },
    {
      "Diet_type": "vegan",
      "Protein(g)": 56.16,
      "Carbs(g)": 254.0,
      "Fat(g)": 103.3
    }
  ],
  "correlation": {
    "Protein(g)": {
      "Protein(g)": 1.0,
      "Carbs(g)": 0.155682,
      "Fat(g)": 0.478012
    },
    "Carbs(g)": {
      "Protein(g)": 0.155682,
```

```
      "Carbs(g)": 1.0,
      "Fat(g)": 0.269059
    },
    "Fat(g)": {
      "Protein(g)": 0.478012,
      "Carbs(g)": 0.269059,
      "Fat(g)": 1.0
    }
  },
  "summary": {
    "total_recipes": 7806,
    "diet_types": [
      "dash",
      "keto",
      "mediterranean",
      "paleo",
      "vegan"
    ],
    "diet_counts": {
      "mediterranean": 1753,
      "dash": 1745,
      "vegan": 1522,
      "keto": 1512,
      "paleo": 1274
    },
    "highest_mean_protein_diet": "keto",
    "highest_mean_protein_g": 101.27,
    "highest_total_protein_diet": "mediterranean",
    "highest_total_protein_g": 177249.89
  },
  "metadata": {
    "execution_time_seconds": 0.027,
    "source": "Local CSV",
    "dataset": "All_Diets.csv",
    "dataset_from_cache": false
  }
}
```

▼ GET /api/recipes · live JSON response


```
{
  "recipes": [
    {
      "Diet_type": "keto",
      "Recipe_name": "Keto Chicken Pot Pie Casserole Recipe",
      "Cuisine_type": "american",
      "Protein(g)": 183.07,
      "Carbs(g)": 66.95,
      "Fat(g)": 330.43
    },
    {
      "Diet_type": "keto",
      "Recipe_name": "Keto Everything Bagel Chicken",
      "Cuisine_type": "american",
      "Protein(g)": 214.84,
      "Carbs(g)": 17.29,
      "Fat(g)": 140.76
    }
  ],
  "count": 2,
  "total": 225,
  "page": 1,
  "limit": 2,
  "total_pages": 113,
```

```
  "diet_type_filter": "keto",
  "search": "chicken",
  "metadata": {
    "execution_time_seconds": 0.002,
    "source": "Local CSV",
    "dataset": "All_Diets.csv",
    "dataset_from_cache": true
  }
}
```

▼ GET /api/clusters · live JSON response


```
{
  "k": 3,
  "features": [
    "Protein(g)",
    "Carbs(g)",
    "Fat(g)"
  ],
  "standardized": true,
  "seed": 42,
  "iterations": 58,
  "inertia": 12825.275678703805,
  "cluster_counts": {
    "0": 1205,
    "1": 5681,
    "2": 920
  },
  "centroids": [
    {
      "cluster": 0,
      "Protein(g)": 228.09124481327768,
      "Carbs(g)": 119.31101244813274,
      "Fat(g)": 265.9383568464727
    },
    {
      "cluster": 1,
      "Protein(g)": 49.97187114944563,
      "Carbs(g)": 100.62553775743703,
      "Fat(g)": 75.44519978876957
    },
    {
      "cluster": 2,
      "Protein(g)": 98.87491304347826,
      "Carbs(g)": 513.0980000000006,
      "Fat(g)": 181.31163043478267
    }
  ],
  "points": [
    {
      "Recipe_name": "Bone Broth From 'Nom Nom Paleo'",
      "Diet_type": "paleo",
      "Protein(g)": 5.22,
      "Carbs(g)": 1.29,
      "Fat(g)": 3.2,
      "cluster": 1
    },
    {
      "Recipe_name": "Paleo Effect Asian-Glazed Pork Sides, A Sweet & Crispy Appetizer",
      "Diet_type": "paleo",
      "Protein(g)": 181.55,
      "Carbs(g)": 28.62,
      "Fat(g)": 146.14,
      "cluster": 0
```

```
    }
  ],
  "metadata": {
    "execution_time_seconds": 0.026,
    "source": "Local CSV",
    "dataset": "All_Diets.csv",
    "dataset_from_cache": true
  }
}
```


---

08 / Run, Demo and Submission Checklist

```
cd Project2/stage1
python3.11 -m venv .venv311
source .venv311/bin/activate
python -m pip install -r requirements-dev.txt
cp local.settings.json.example local.settings.json
python -m pytest -q
python -m flake8 . --exclude=.venv,.venv311
func start --port 7071
# in a second terminal:
python verify_http.py
```

The last two commands need Azure Functions Core Tools v4 (`npm i -g azure-functions-core-tools@4`). No cloud account or key is required for any endpoint.

1. pytest all green and flake8 clean — done, see Figure S1.
2. Start the Functions host and open /api/health and /api/insights — done, see Figures S2, S4 and S5.
3. /api/recipes?diet_type=keto&search=chicken&page=1&limit=5 for filtering, search and paging — done, see Figure S6; verify_http.py asserts that page 2 differs from page 1.
4. /api/clusters?k=3 for cluster sizes, centroids and points — done, see Figure S7.
5. The code is committed and merged to main, and the API contract is handed to Person 2; Person 3 takes on Blob Storage, cloud deployment and the final report.

> **Group submission:** working Azure back-end and front-end links, the GitHub repository, a PDF/Word document with screenshots covering each rubric row, and a roughly 10-minute video featuring every member. The project already holds cloud deployment screenshots and the group report; this round did not re-check whether the remote services are still online, and Stage 1 did its verification locally without any cloud release or configuration change. This HTML is the Stage 1 summary and can be printed to PDF and folded into the group report.

Sources: Project2/division.md, spec/submission-requirements.md, the existing backend and tests, and the Stage 1 measurements taken here. The original deployment folder is unchanged, and the existing deploy.sh still targets backend/.

Part 2

Stage 2 — Frontend Dashboard

Contributor: Ethan Bayarsaikhan

CPSY 300 · CLOUD COMPUTING · PROJECT 2

Stage 2 Frontend Summary

Interactive Dashboard & Data Visualization

Author: Ethan Bayarsaikhan (Person 2) · Stage 2 | Verified: 2026-11-06

This page summarises the Stage 2 dashboard and its evidence. Person numbering follows the team's division of work.

01 / Delivery at a Glance

4	11	3	0
visualizations	browser screenshots	API action buttons	build steps (Chart.js via CDN)

Stage 2 builds the whole dashboard on top of Stage 1's merged main: a static site in plain HTML, CSS and JavaScript with Chart.js from a CDN, so there is no build step and the folder deploys to an Azure Static Web App as-is. A single fetch helper with one `API_BASE_URL` reads the four endpoints and fills four charts, a searchable recipe table and the status badges.

> **Status:** Stage 2 is complete and evidenced against the local Function host at `http://localhost:7071`. All four visualizations, the search box, the diet filter, the three action buttons and Previous/Next pagination work; the execution-time badge shows each call's server time and browser round trip, and a red error banner appears when the API is unreachable. Eleven browser screenshots are archived in `stage2/screenshots/`. Moving to Azure is Stage 3 — only the one line in `config.js` changes.

02 / Stage 2 Requirements Checklist

Requirement from the division of work	Where it lives and how it is evidenced	Status
Page layout and styling to match the UI skeleton	index.html, styles.css	Done

Requirement from the division of work	Where it lives and how it is evidenced	Status
One fetch helper with a single API_URL setting	api.js `apiGet()` , config.js `API_BASE_URL`	Done
Bar chart (average macros)	charts.js `renderBarChart` , canvas `#barChart`	Verified
Pie chart (recipes per diet)	charts.js `renderPieChart` , canvas `#pieChart`	Verified
Correlation heatmap	charts.js, `#heatmap` CSS grid	Verified
Scatter chart coloured by cluster	charts.js, canvas `#scatterPlot` , `/clusters?k=3`	Verified
Search box, diet dropdown and the three action buttons	index.html controls, recipes.js	Verified
Recipe table with Previous / Next pagination	recipes.js, `#recipesBody` , `#prevBtn / #nextBtn`	Verified
Execution-time badge, plus an error banner when the API is down	api.js `showExecutionTime / showError` , `#executionTime` , `#errorBanner`	Verified

Checked against the Stage 2 items in `division.md` . One `/insights` request feeds three of the four charts (bar, pie and heatmap); the scatter plot uses `/clusters` and the table uses `/recipes` .

03 / Structure and Files

Page open → apiGet() fetch helper → /insights · /recipes · /clusters → Charts, table, badges

File	Purpose
index.html	Page layout, following the course UI skeleton: status bar, four chart cards, controls and the recipe table
styles.css	All styling; colours and sizes match the skeleton
config.js	`API_BASE_URL` — the only line that changes between local and Azure
api.js	`apiGet()` fetch helper: error handling, the red error banner and the execution-time badge
charts.js	Bar, pie and scatter charts with Chart.js; the heatmap is an HTML/CSS grid
recipes.js	Recipe table, search, diet filter and Previous/Next pagination
app.js	Loads everything on page open and wires up the buttons

`config.js` sets `API_BASE_URL` to `http://localhost:7071/api` for local runs; for the live site Stage 3 changes it to the Azure Function URL ending in `/api` with no trailing slash. All API text is inserted with

`textContent`, never `innerHTML`, so dataset values can never run as HTML.

---

04 / Dashboard Walkthrough

The eleven screenshots below are the Stage 2 run evidence against the local backend. They are embedded, so this page can be sent on its own.

Frontend dashboard

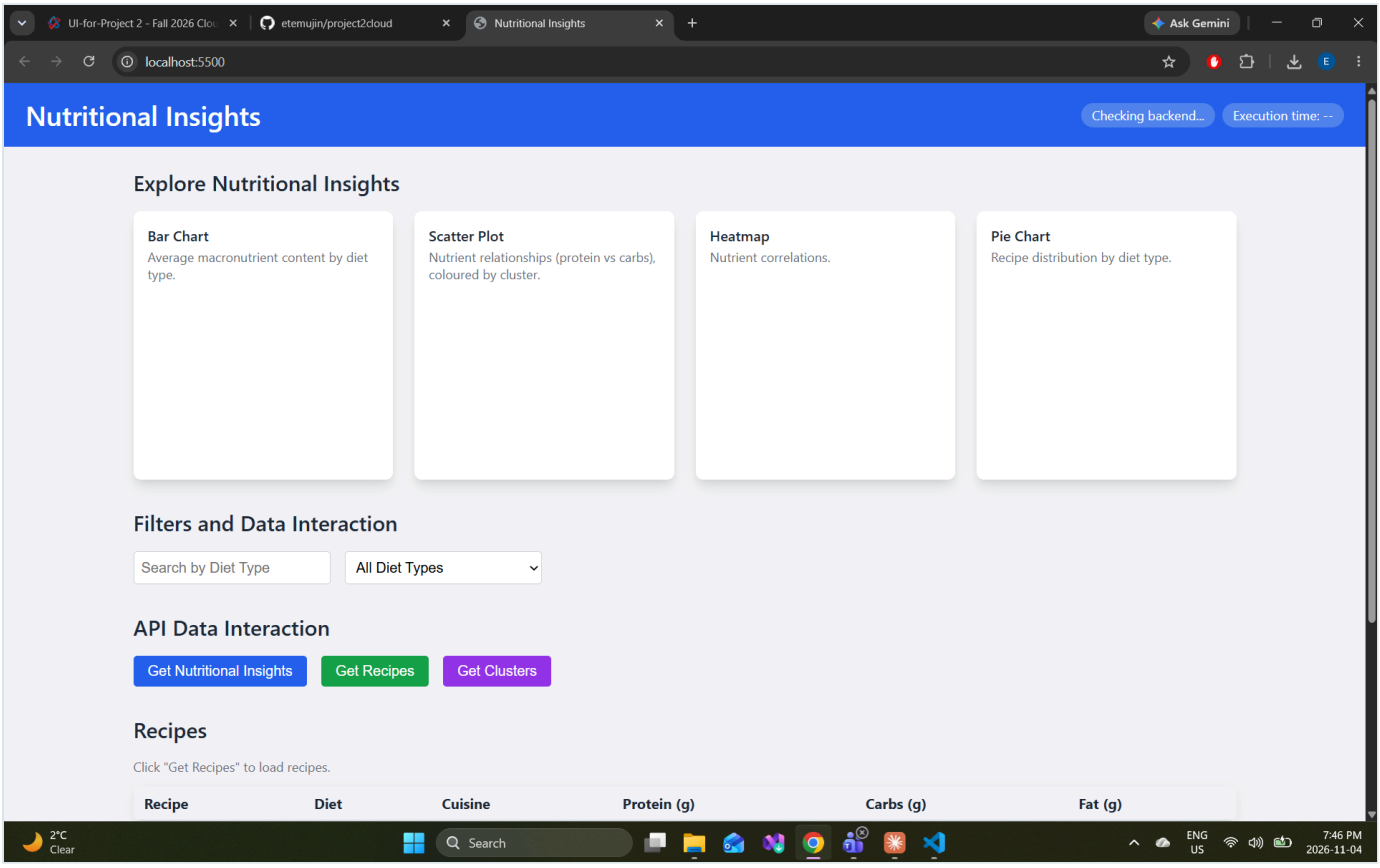


Figure 1 — Dashboard layout. The header carries backend-health and execution-time badges; below are four chart cards, the search and diet filters, three action buttons and the recipe table. (`stage2/screenshots/sc 3 ui.png`)

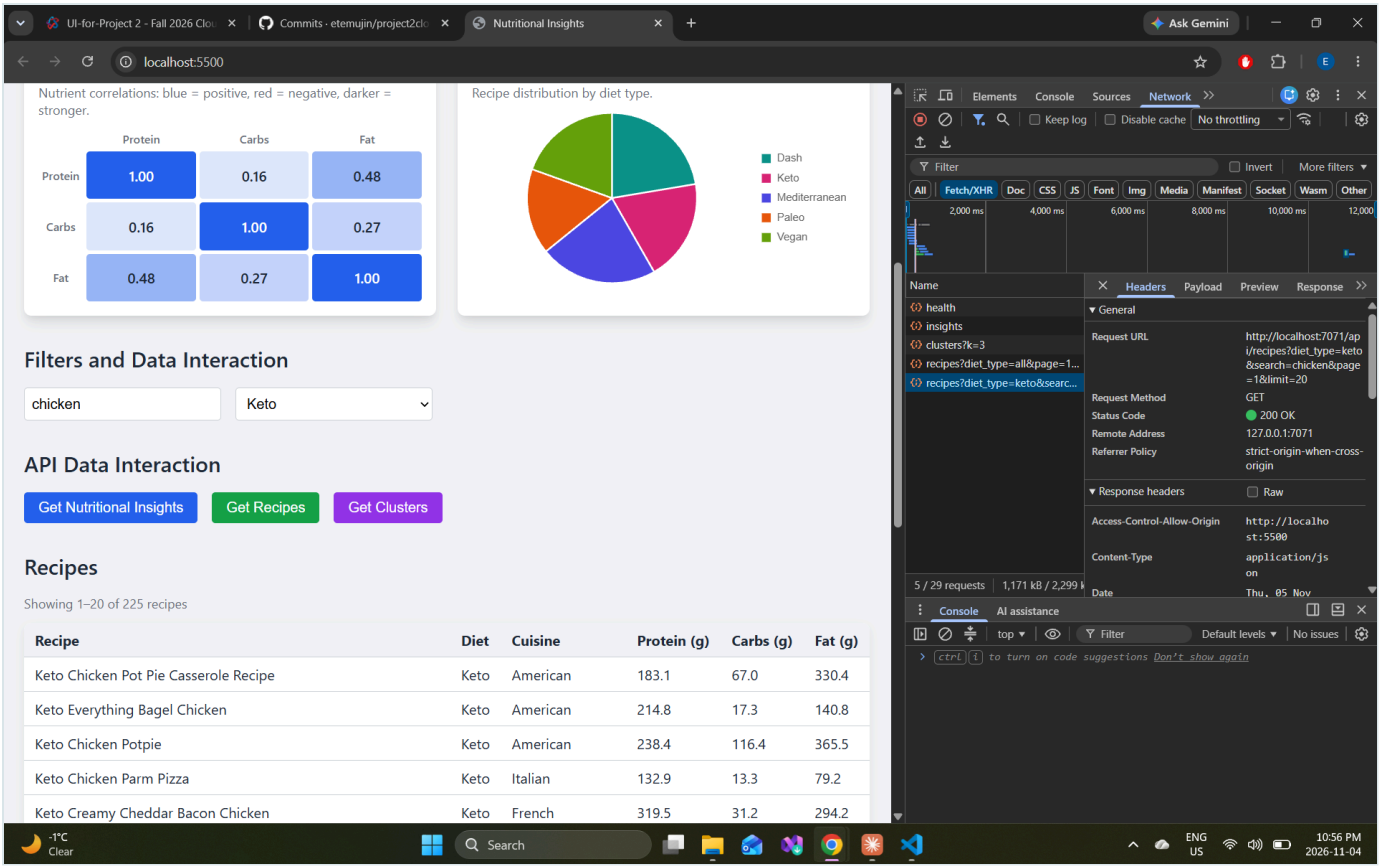


Figure 2 — Search, filter and recipe table. Searching "chicken" with the Keto filter returns 225 recipes, shown 20 per page; DevTools shows the request URL and the CORS header allowing http://localhost:5500. (`stage2/screenshots/sc 9 ui.png`)

Data visualization

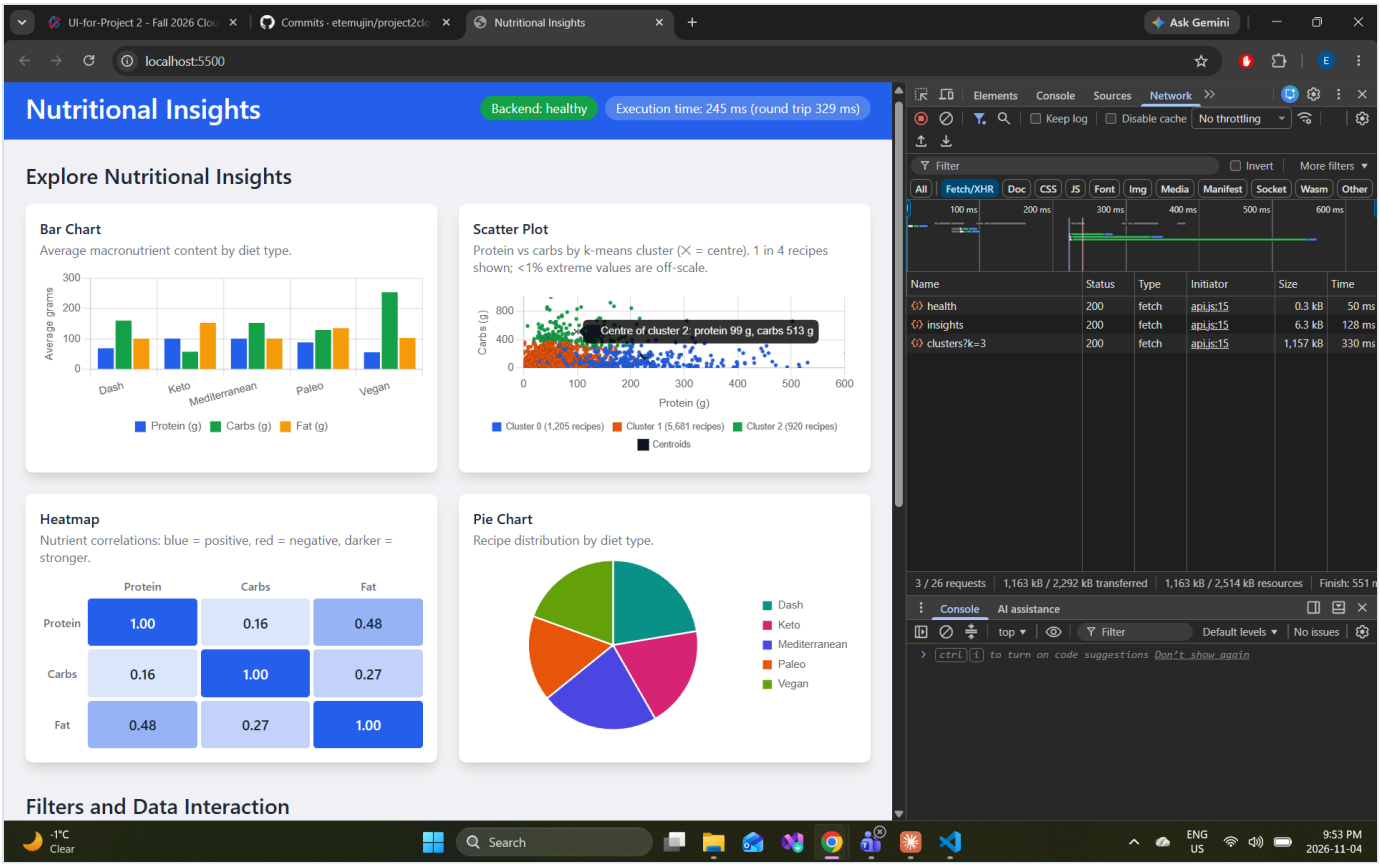


Figure 3 — All four visualizations drawn from the API; the Network tab shows the health, insights and clusters requests returning 200. (`stage2/screenshots/sc 8 ui.png`)

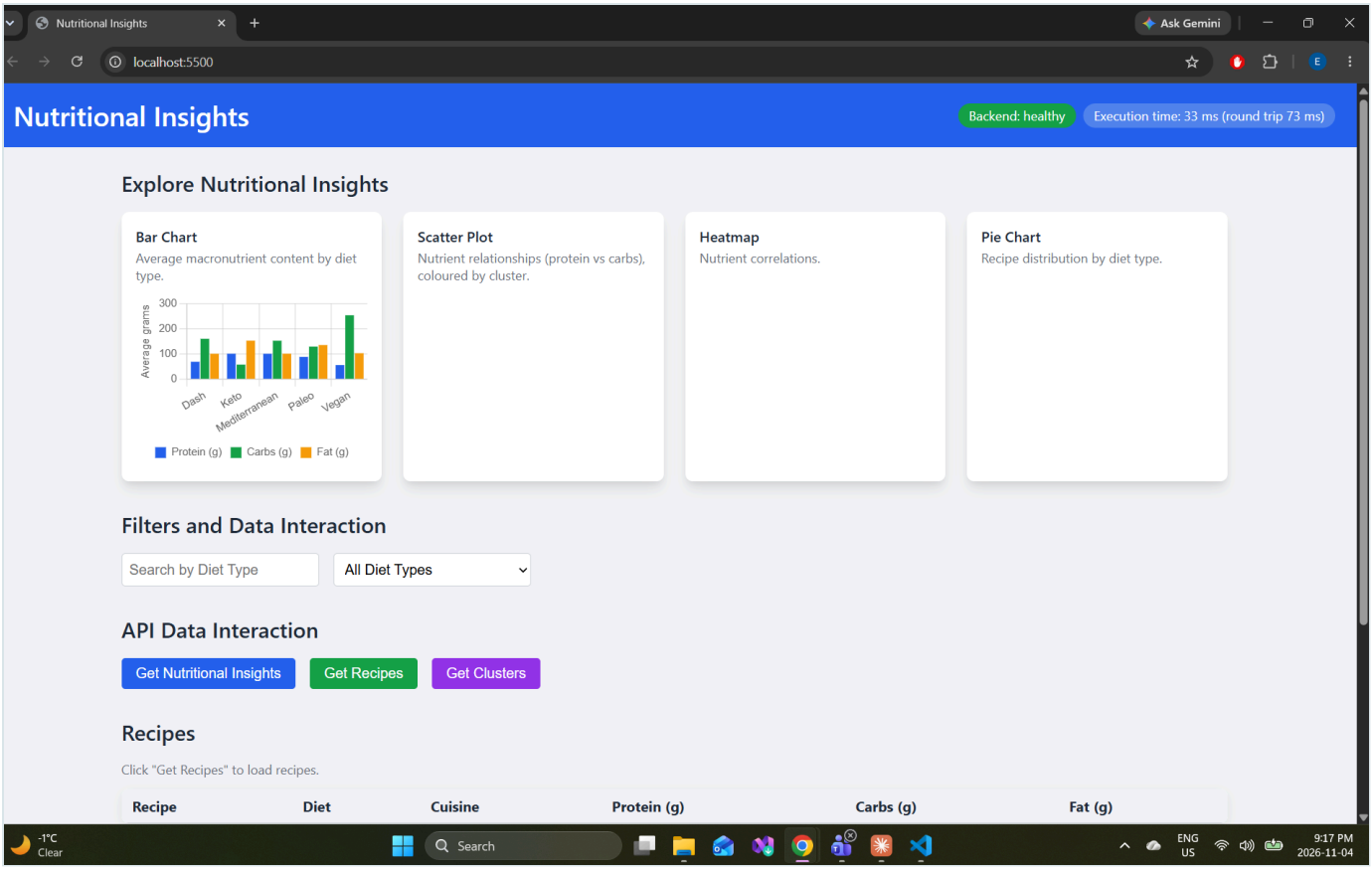


Figure 4 — Bar chart: average protein, carbs and fat per diet type, from /insights. Vegan has the most carbs and keto the most fat. (`stage2/screenshots/sc 5 ui.png`)

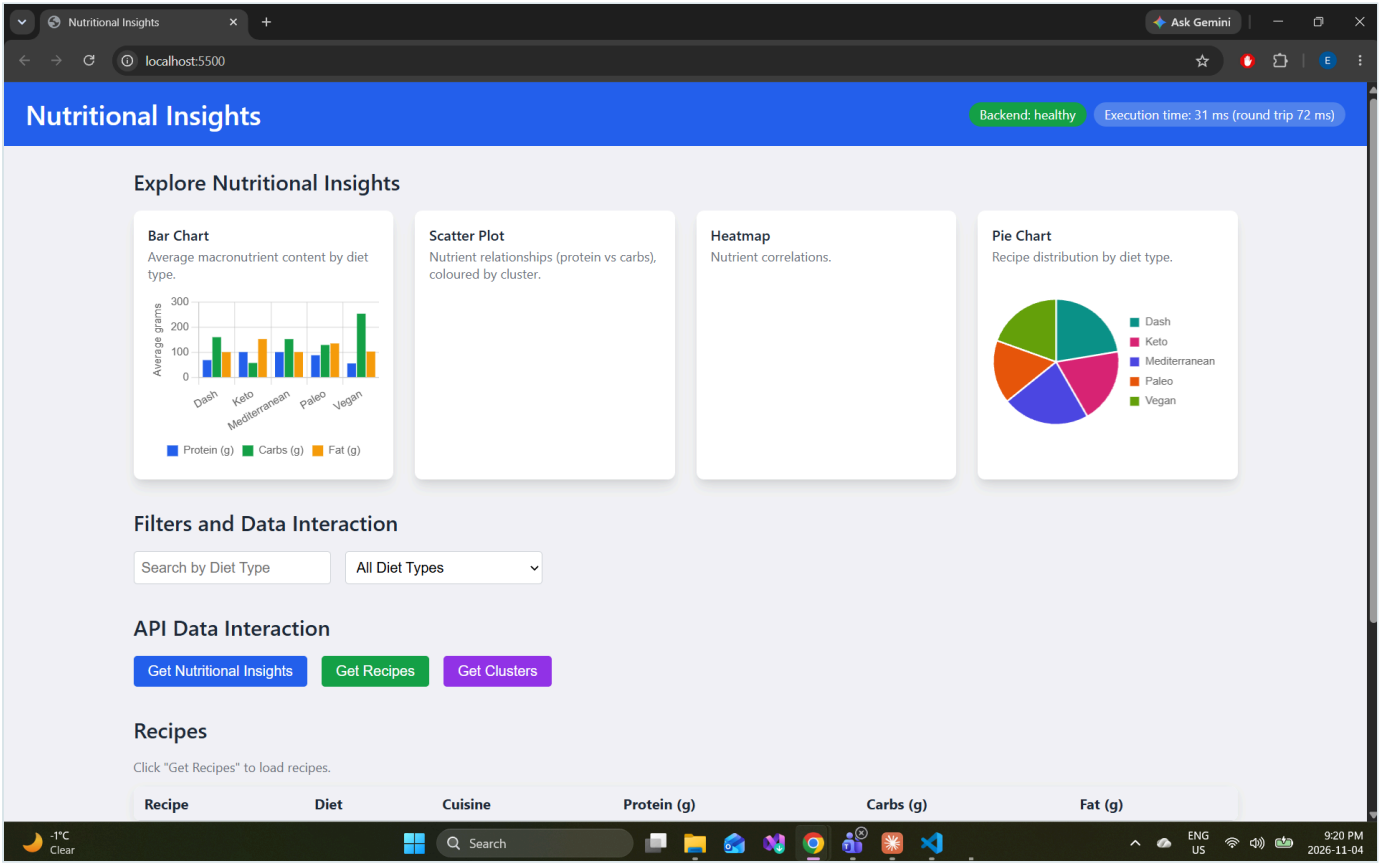


Figure 5 — Pie chart: recipe count per diet type, 7,806 recipes in total. (`stage2/screenshots/sc 6 ui.png`)

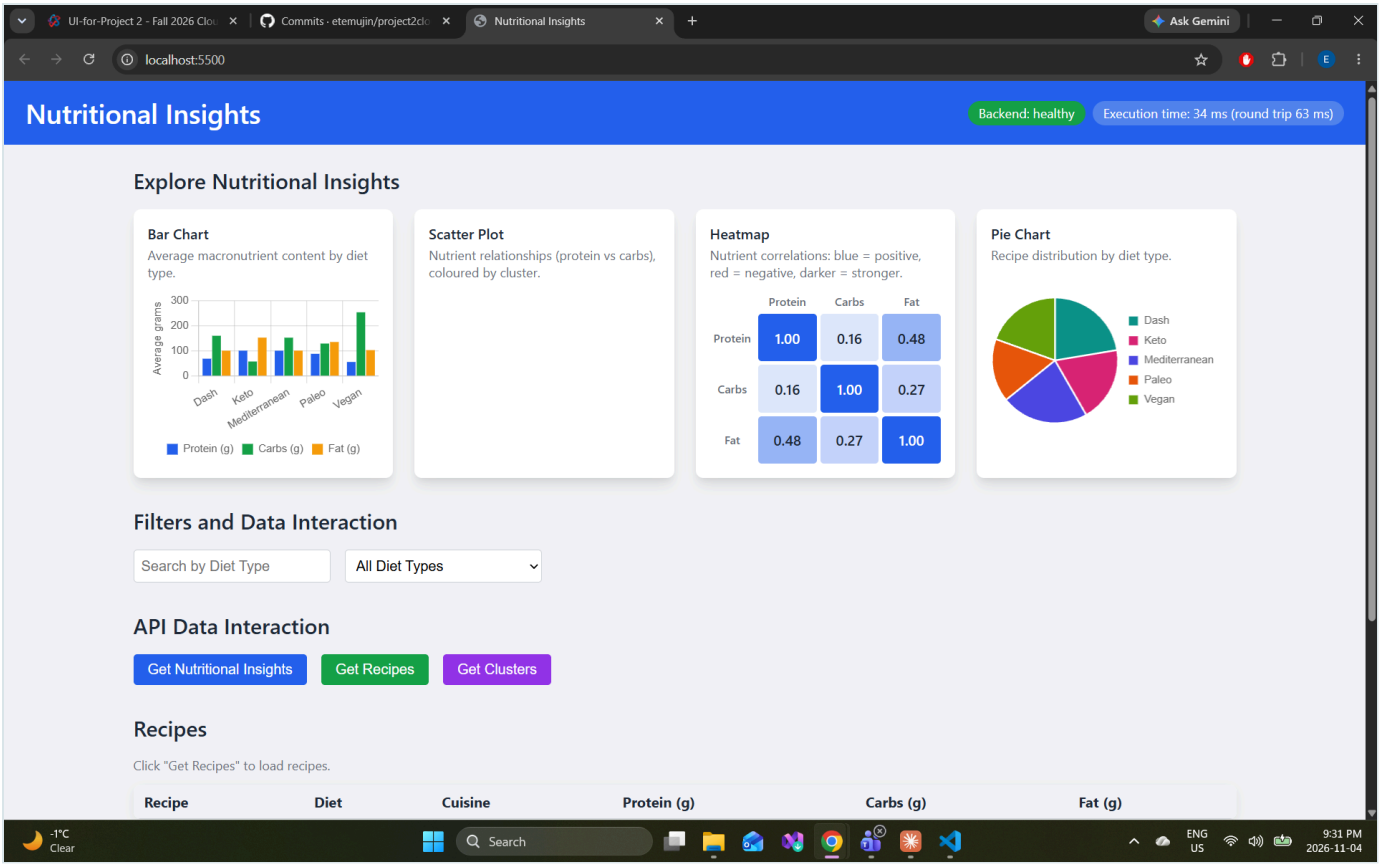


Figure 6 — Correlation heatmap for protein, carbs and fat; protein and fat are the most related (0.48). (`stage2/screenshots/sc7 ui.png`)

Integration

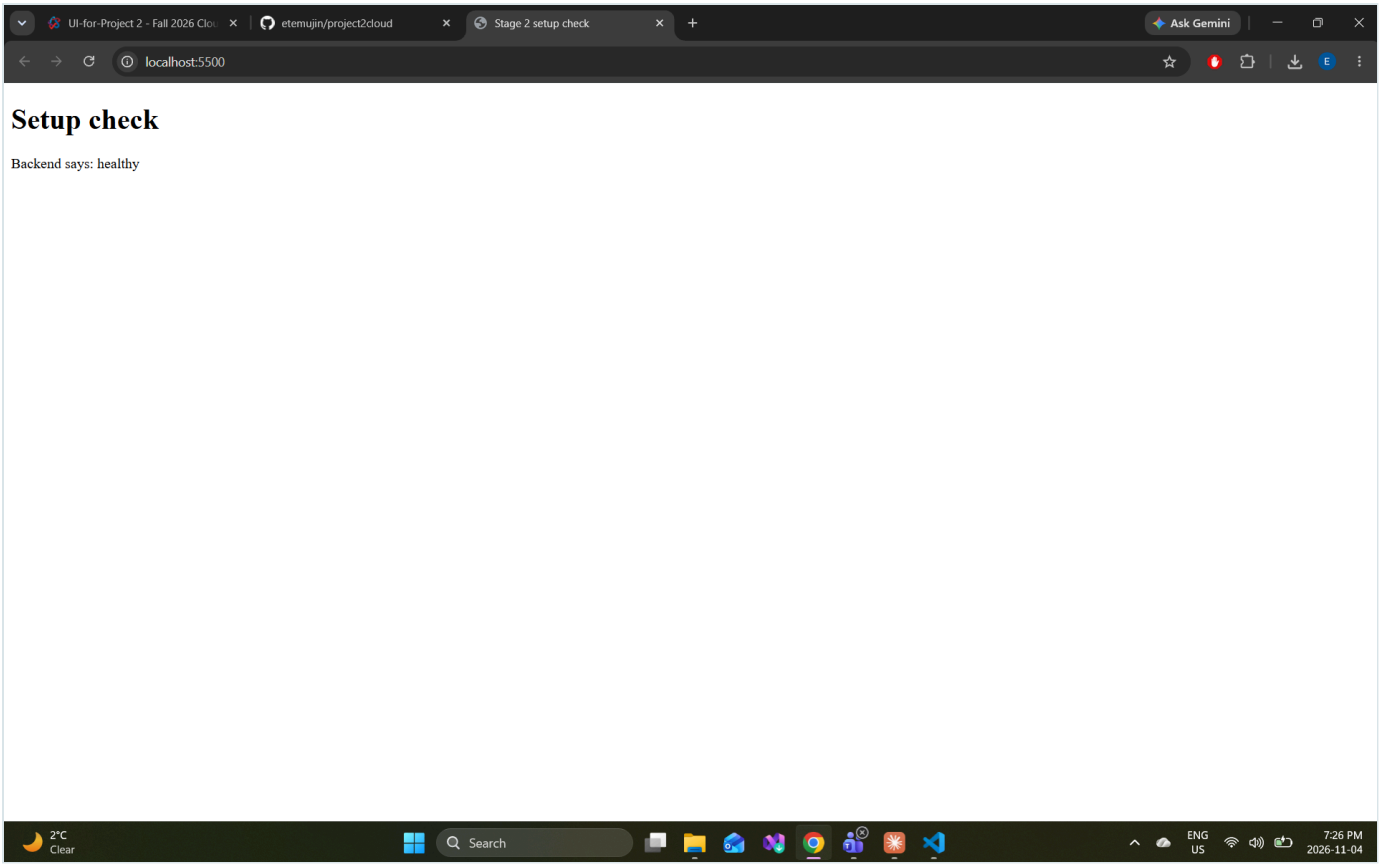


Figure 7 — Setup check: a test page on localhost:5500 calls /api/health and shows its answer, "healthy".
(`stage2/screenshots/sc 1 ui.png`)

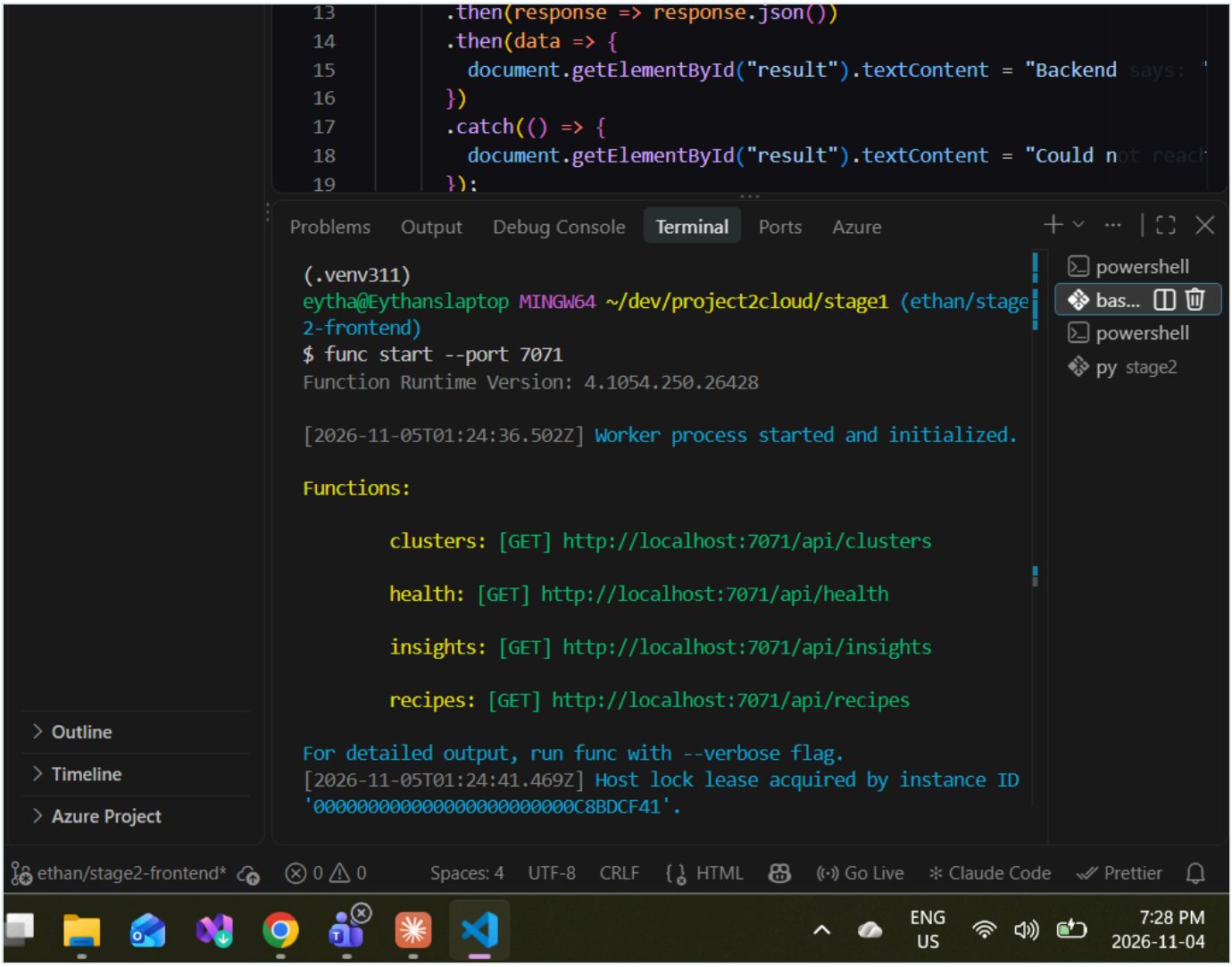


Figure 8 — func start --port 7071 brings up the Functions host and registers the clusters, health, insights and recipes routes.
(`stage2/screenshots/sc 2 ui.png`)

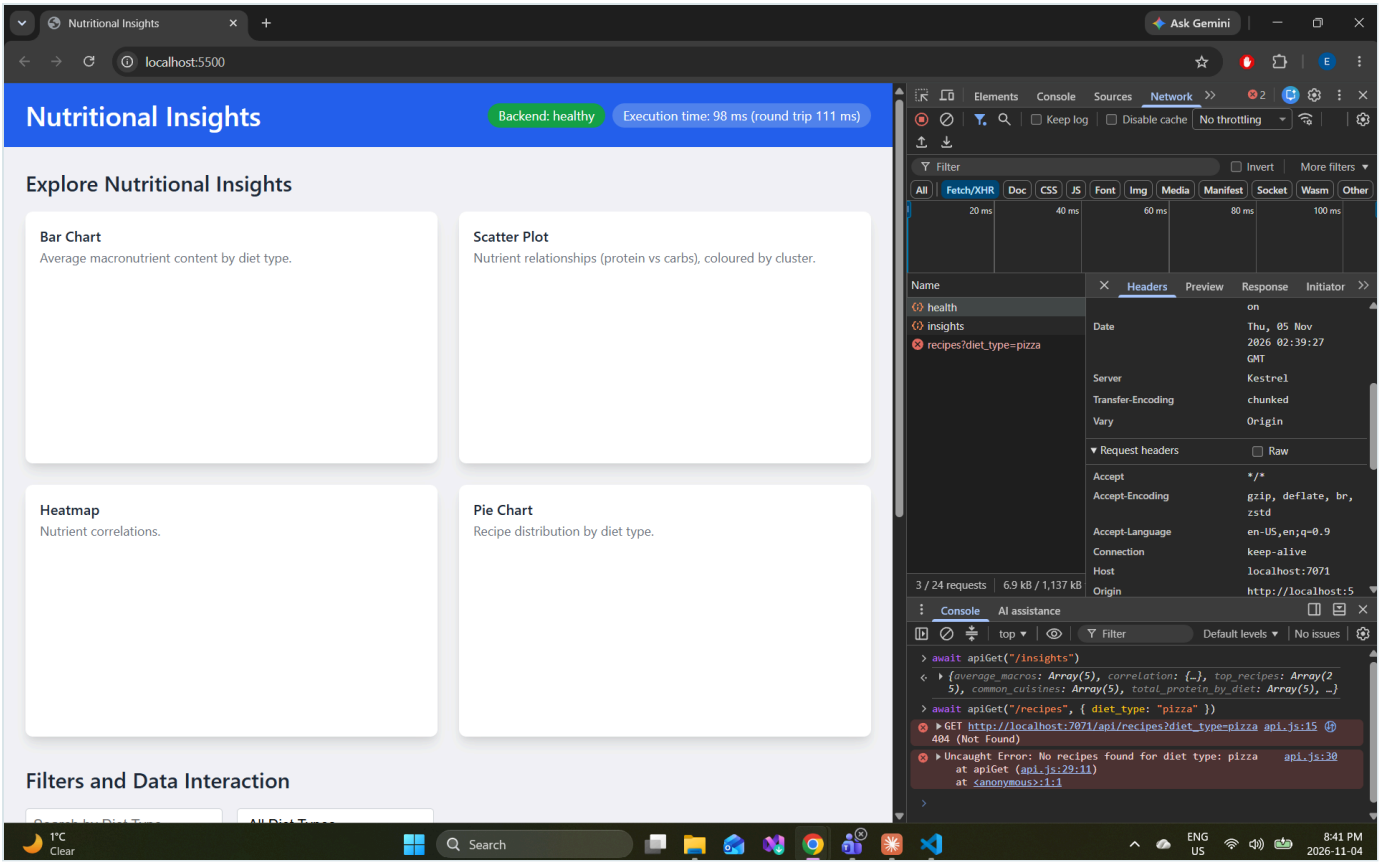


Figure 9 — Fetch helper and API errors: the health badge turns green; apiGet() is tested in the console, and an unknown diet returns a 404. (`stage2/screenshots/sc 4 ui.png`)

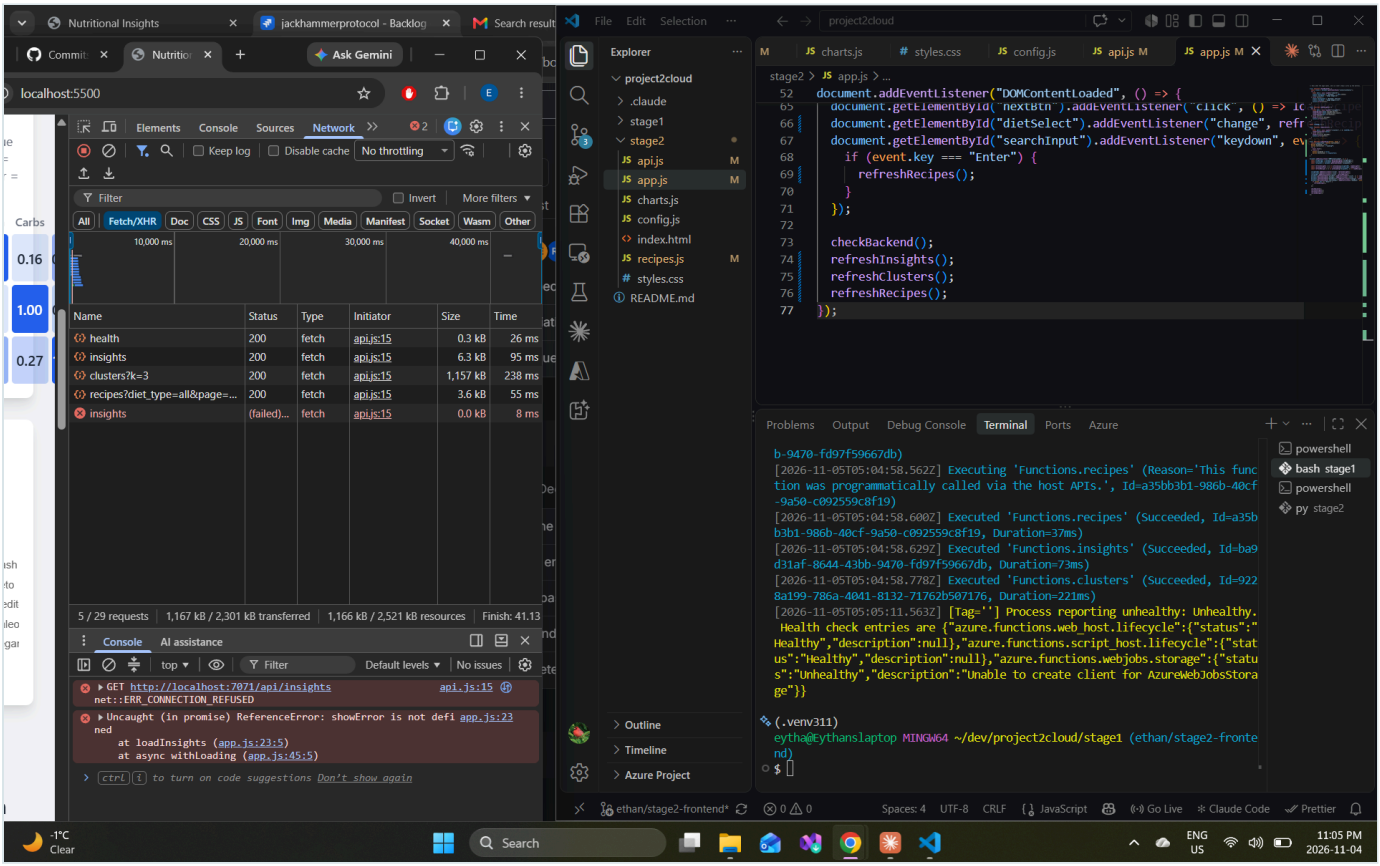


Figure 10 — Backend stopped: the insights request fails with ERR_CONNECTION_REFUSED; the final version shows a red error banner instead. (`stage2/screenshots/sc 10 ui.png`)

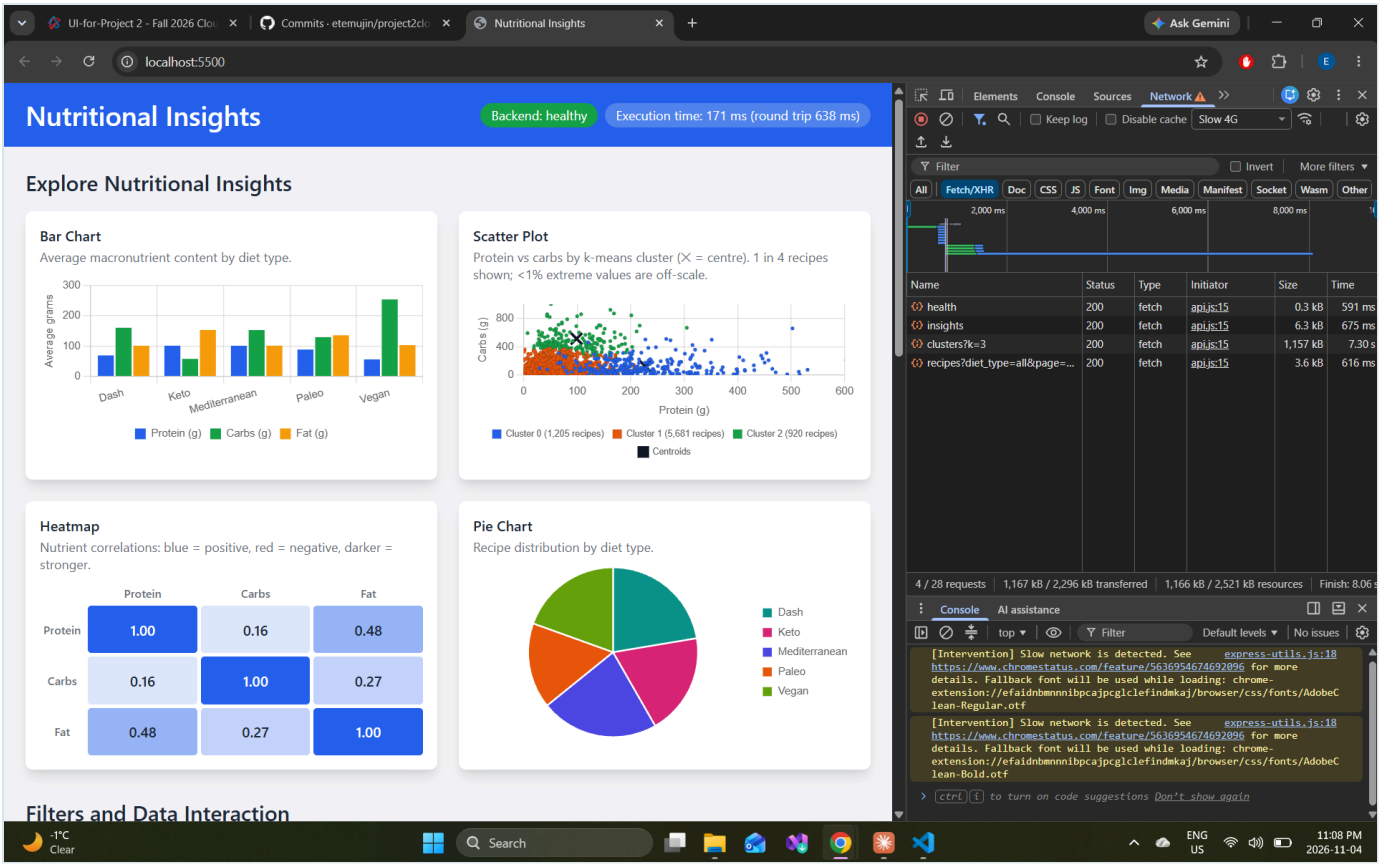


Figure 11 — Slow 4G: on a throttled connection every request still succeeds; the 1,157 kB clusters response takes 7.3 s. (`stage2/screenshots/sc 11 ui.png`)

---

05 / Charts and Controls

Four visualizations

- **Bar** average macronutrients per diet type, from `/insights` .
- **Pie** recipe count per diet type, from `summary.diet_counts` .
- **Heatmap** nutrient correlations as a CSS grid (Chart.js has no heatmap type); opacity is correlation strength, blue positive and red negative, with protein and fat the most related (0.48).
- **Scatter** protein vs carbs coloured by k-means cluster from `/clusters?k=3` , centres marked ×.

Controls

- Search box and a diet dropdown (All, Dash, Keto, Mediterranean, Paleo, Vegan).
- Get Nutritional Insights / Get Recipes / Get Clusters buttons; each shows “Loading…” and is disabled while its request runs.
- Recipe table with Previous / Next pagination, 20 rows per page.
- Execution-time badge showing the Function’s own time and the browser round trip, labelled by endpoint.
- A red error banner when the API cannot be reached; it hides again after the next success.

Design choices. The scatter plot draws every 4th of the 7,806 points (about 1,950) so it stays readable, while the legend still shows each cluster's full count; its axes are fixed at 0–600 g protein and 0–1,000 g carbs, which keeps 99.2% of recipes on scale instead of squashing them into one corner.

---

06 / Running Locally

```
# 1) start the backend (Git Bash)
cd stage1
source .venv311/Scripts/activate
func start --port 7071

# 2) serve the dashboard in a second terminal
cd stage2
py -3.11 -m http.server 5500

# 3) open the dashboard
http://localhost:5500
```

Port 5500 matters: the backend's local CORS setting only allows `http://localhost:5500` and `http://127.0.0.1:5500`. Opening `index.html` straight from the file system will not work. Handing off to Stage 3 is a one-line change: point `API_BASE_URL` at the live Function App and add the Static Web App's origin to the Function App's CORS list.

Part 3

Stage 3 — Azure Cloud Deployment

Contributor: Justin Norman-Rance

CPSY 300 · CLOUD COMPUTING · PROJECT 2

Stage 3 Cloud Deployment Summary

Azure Blob Storage, Functions & Static Web Apps

Author: Justin Norman-Rance (Person 3) · Stage 3 | Verified: 2026-11-06

This page summarises the Stage 3 Azure deployment and its live evidence. Person numbering follows the team's division of work.

01 / Delivery at a Glance

2	3	16	1
live URLs (API + dashboard)	core Azure services	deployment screenshots	resource group, one-command teardown

Stage 3 of `division.md` takes the Stage 1 backend and the Stage 2 dashboard to the cloud. The dataset moves from a local CSV to a **private Azure Blob Storage** container, the API runs on a consumption-plan **Azure Function App**, and the dashboard is served from **Azure Static Web Apps** — all inside one resource group, created and removed by `deploy.sh` and `teardown.sh`.

> **Status:** Deployment verified on 2026-10-20. The live dashboard and the API both respond; the header shows a green "API Connected" pill and every data response carries `metadata.execution_time_seconds`. The storage connection string lives only in the Function App settings — never committed, never printed — and CORS is scoped at the host level to the dashboard origin only. The Static Web App is in `eastus2` because Static Web Apps is not offered in `canadacentral`, where the rest of the resources live.

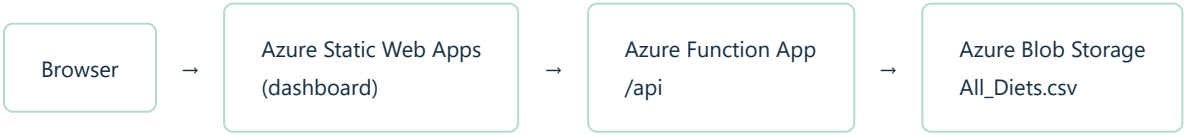
02 / Stage 3 Requirements Checklist

Requirement from the division of work	Where it lives and how it is evidenced	Status
Resource group, storage account, private container, upload the CSV	`deploy.sh` steps 1–2; `portal-resource-group.png`, `portal-storage-container.png`	Verified
Function App + settings; switch data loading from the local file to Blob Storage	`backend/function_app.py` (`BlobServiceClient`, `source: "Azure Blob Storage"`); app settings `AZURE_STORAGE_CONNECTION_STRING` / `BLOB_CONTAINER_NAME` / `BLOB_NAME`	Verified
Deploy the backend and test the live endpoints	`deploy.sh` step 4 + smoke test; `api-health-browser.png`, `api-insights-response.png`, `api-recipes-response.png`	Verified
Static Web App, set the API URL to the live Function, deploy the frontend	`deploy.sh` steps 5–6; `portal-static-web-app-essentials.png`, `dashboard-live.png`	Verified
Scope CORS so only the dashboard can call the API	`deploy.sh` step 7 removes the wildcard and adds only the SWA origin; the function code emits no CORS headers; `cli-cors-and-timing.png`	Verified
Write `deploy.sh` and `teardown.sh`	`deploy.sh` (idempotent, re-runnable), `teardown.sh` (deletes the whole resource group)	Done
Azure portal and live-site screenshots for every rubric row	`docs/screenshots/`, 16 images	Done
Write the document: architecture, screenshots, URLs, challenges	`docs/phase2-documentation.md`, `docs/phase2-analysis.md`; this summary	Done

The checklist follows the Stage 3 section of `division.md`. The deployed cloud API exposes four endpoints — `/api/health`, `/api/insights`, `/api/recipes` and `/api/clusters` — the same contract as the Stage 1 local backend, now served from Azure Blob Storage.

---

03 / Architecture



Why this shape

- **Serverless compute.** The workload is bursty and read-only, so a consumption-plan Function App costs nothing while

Regions

Everything lives in one resource group, `cpsy300-nutritional-insights-rg`, in `canadacentral` — except the Static Web App,

idle and needs no VM or container.

- **Hosting separate from compute.** Static assets are served by a CDN-backed host; only data requests reach the Function, so the dashboard stays fast and the Function scales on its own.
- **Private storage, no public blob URL.** The browser never talks to Blob Storage. The Function is the only reader and authenticates with a connection string in app settings, so the container stays private.
- **Stateless Function, cached dataset.** No database: the cleaned DataFrame is cached in worker memory and invalidated by the blob's ETag.

which is in `eastus2` because Static Web Apps is not available in `canadacentral`. The split is an explicit, overridable variable in `deploy.sh`, not an inconsistency.

> The browser reaches the API cross-origin over HTTPS; host-level CORS allows only the Static Web App origin, so no other site can call the Function.

04 / Azure Services Used

Service	Configuration	Role
Resource Group	`cpsy300-nutritional-insights-rg`, `canadacentral`	One logical container for every resource, so the whole environment deploys, audits and deletes as a unit.
Storage Account	StorageV2, Standard_LRS, TLS 1.2 minimum, public blob access disabled	Durable, cheap object store for the dataset; also backs the Function App's own `AzureWebJobsStorage`.
Blob container `datasets`	Private; holds `All_Diets.csv`	Keeps the dataset out of the repo and the deployment package; private means no anonymous URL exists.
Function App	`cpsy300-nutrition-api`, Python 3.11, Functions v4, Linux consumption	Serverless HTTP API running the cleaning and aggregation code on demand; pay-per-execution and automatic scale-out.
Static Web App	`cpsy300-nutrition-dashboard`, Free SKU, `eastus2`, created unlinked	Public HTTPS hosting for the dashboard with a default hostname; unlinked so no GitHub token is needed.
Function App settings	`AZURE_STORAGE_CONNECTION_STRING`, `BLOB_CONTAINER_NAME`, `BLOB_NAME`	Externalises all configuration and the one secret; nothing environment-specific is hard-coded.
CORS configuration	Host-level allowed origins, scoped to the SWA origin	Lets the dashboard call the API from its own origin without opening the API to every website.
Application Insights	Provisioned with the Function App	Captures logs and exceptions — where internal error detail goes instead of into HTTP responses.

05 / Deployment Pipeline

One command, `./deploy.sh`, runs seven idempotent steps. Re-running it updates the existing resources and reuses the storage account already in the group instead of creating a second one.

```
[1/7] Resource group     cpsy300-nutritional-insights-rg   (canadacentral)
[2/7] Storage + dataset  cpsy300nutrition  ->  container "datasets" (private)  ->  upload All_Diets.csv
[3/7] Function App       cpsy300-nutrition-api  (Python 3.11, Functions v4, consumption)
                         app settings: connection string, container name, blob name
[4/7] Publish backend    func azure functionapp publish   ->  smoke-tested
[5/7] Static Web App     cpsy300-nutrition-dashboard  (Free, eastus2, unlinked)
[6/7] Build + deploy FE  inject the live API URL into a throwaway build/, then swa deploy
[7/7] Scope CORS         remove '*', allow only the dashboard origin, then smoke-test

Function App : https://cpsy300-nutrition-api.azurewebsites.net/api
Dashboard    : https://salmon-ocean-0ce94150f.4.azurestaticapps.net/
```

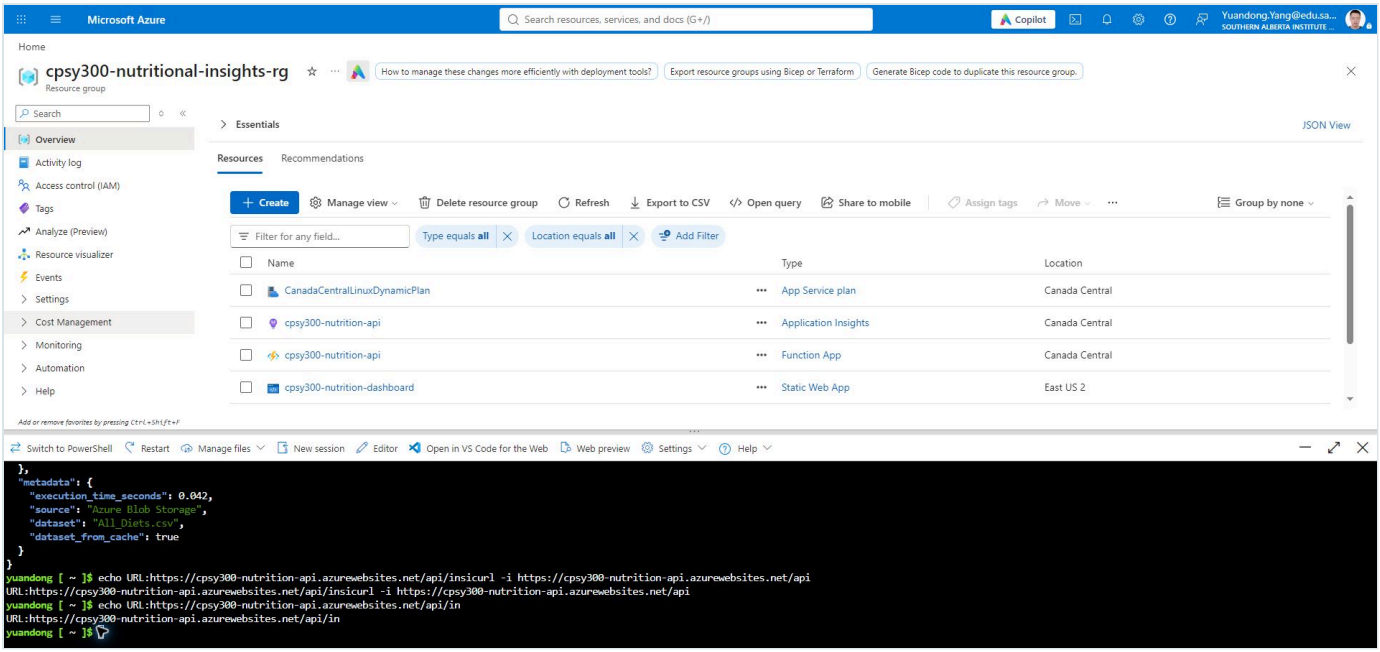
The storage connection string is written straight into the Function App settings and **never printed**, so it cannot leak into terminal scrollback, a screenshot or a screen recording; `deploy.sh` prints the `az` command to retrieve it on demand instead. Every name, region and runtime version is an overridable environment variable.

---

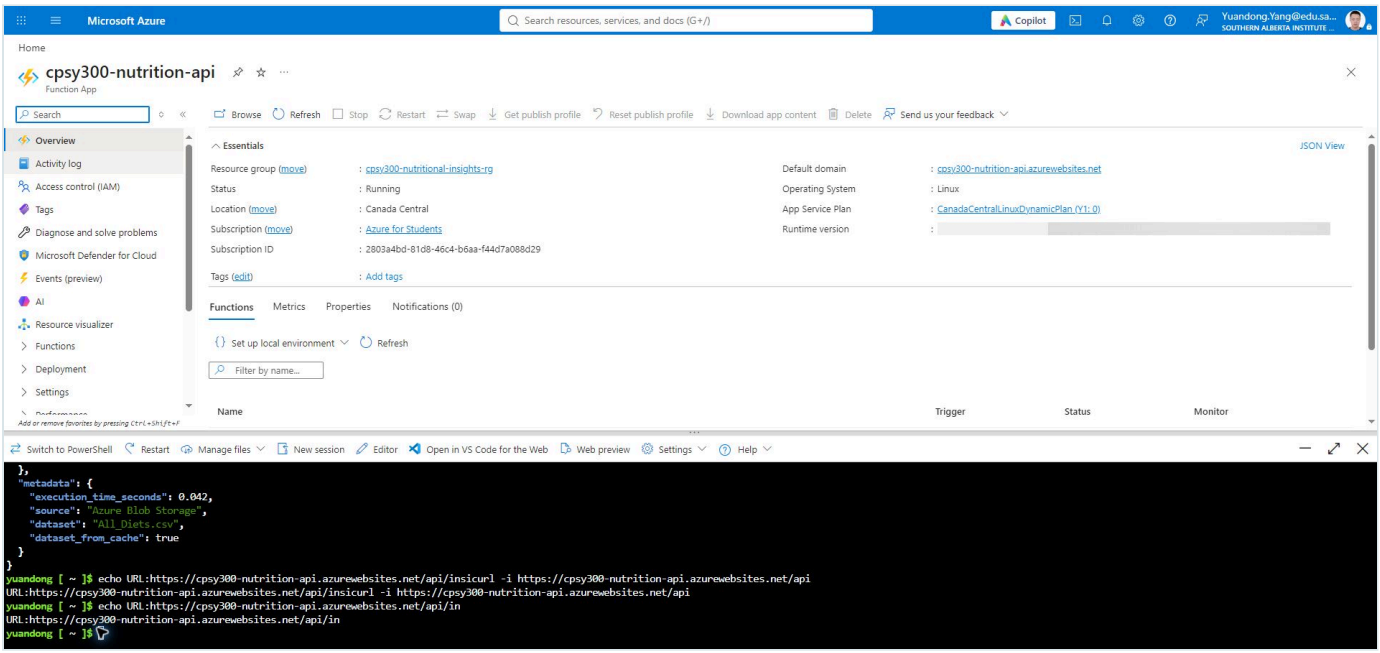
06 / Live Evidence

Captured from the live Azure deployment on 2026-10-20. Portal captures show the current resource state; the dashboard capture shows the public URL and the live API status. All images are archived in `docs/screenshots/`.

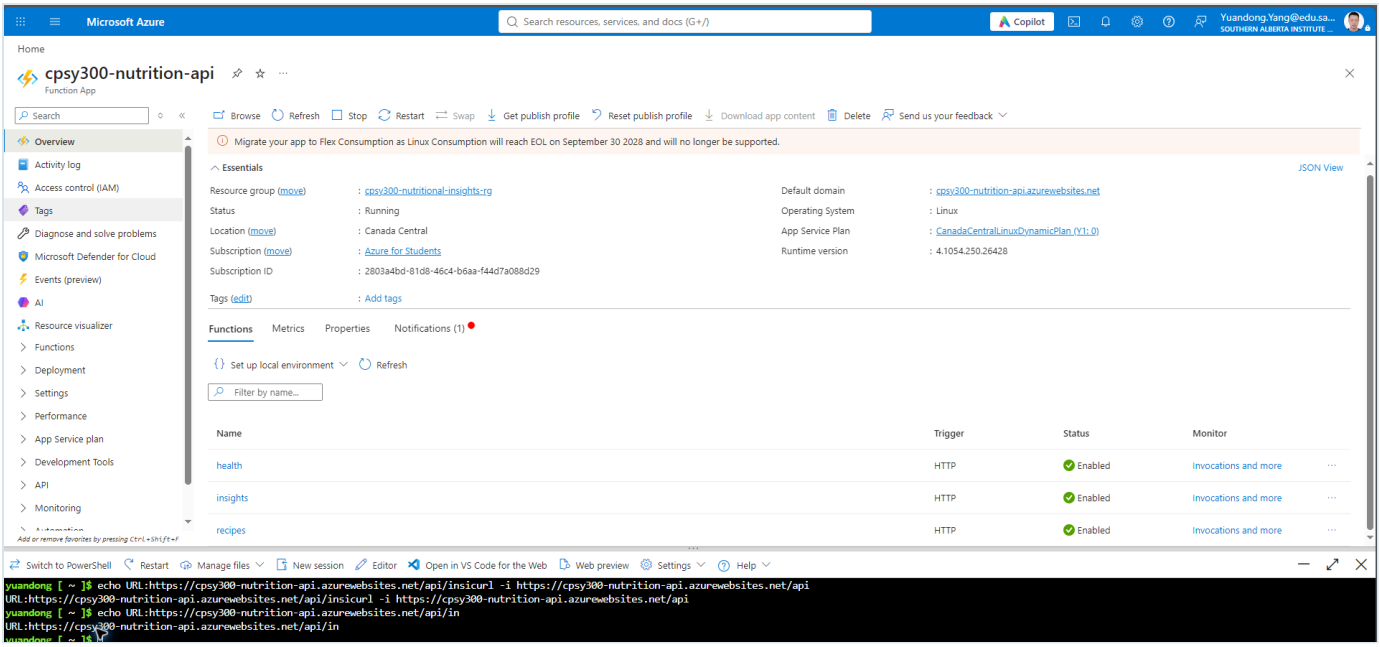
Azure Portal



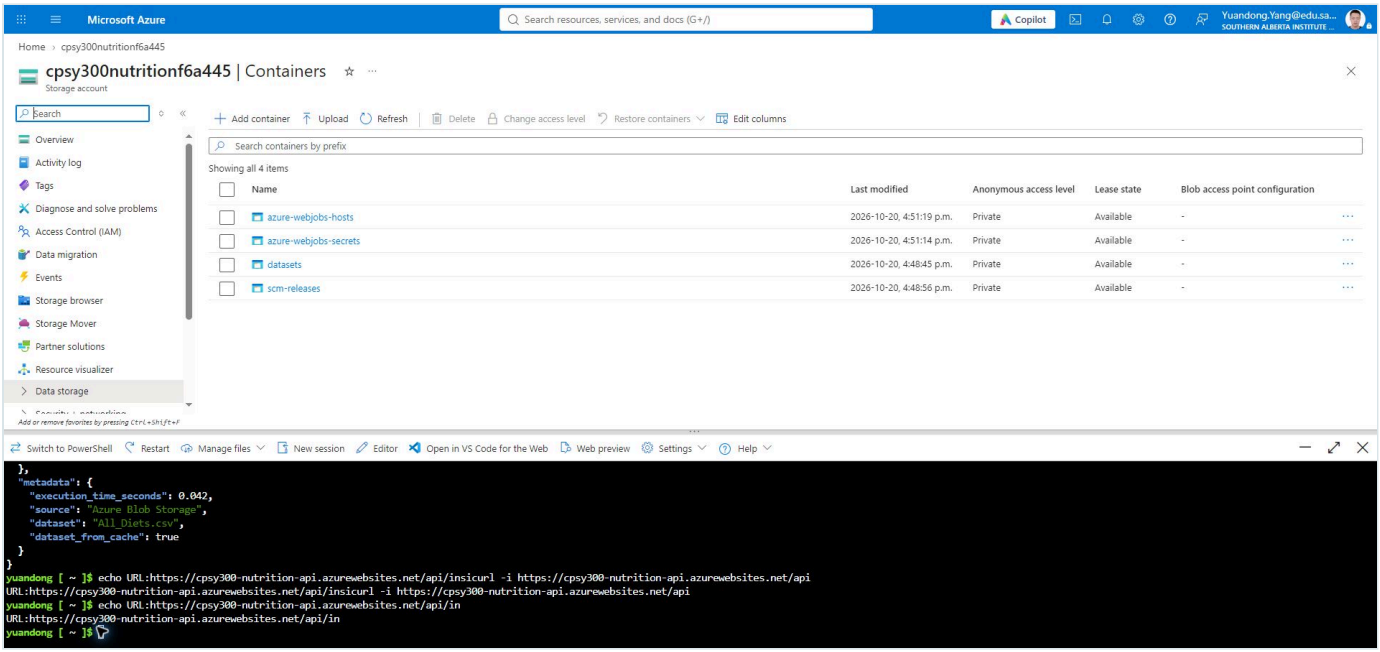
Azure portal — resource group `cpsy300-nutritional-insights-rg`: the storage account, Function App, App Service plan, Application Insights and Static Web App, with their types and regions (`docs/screenshots/portal-resource-group.png`)



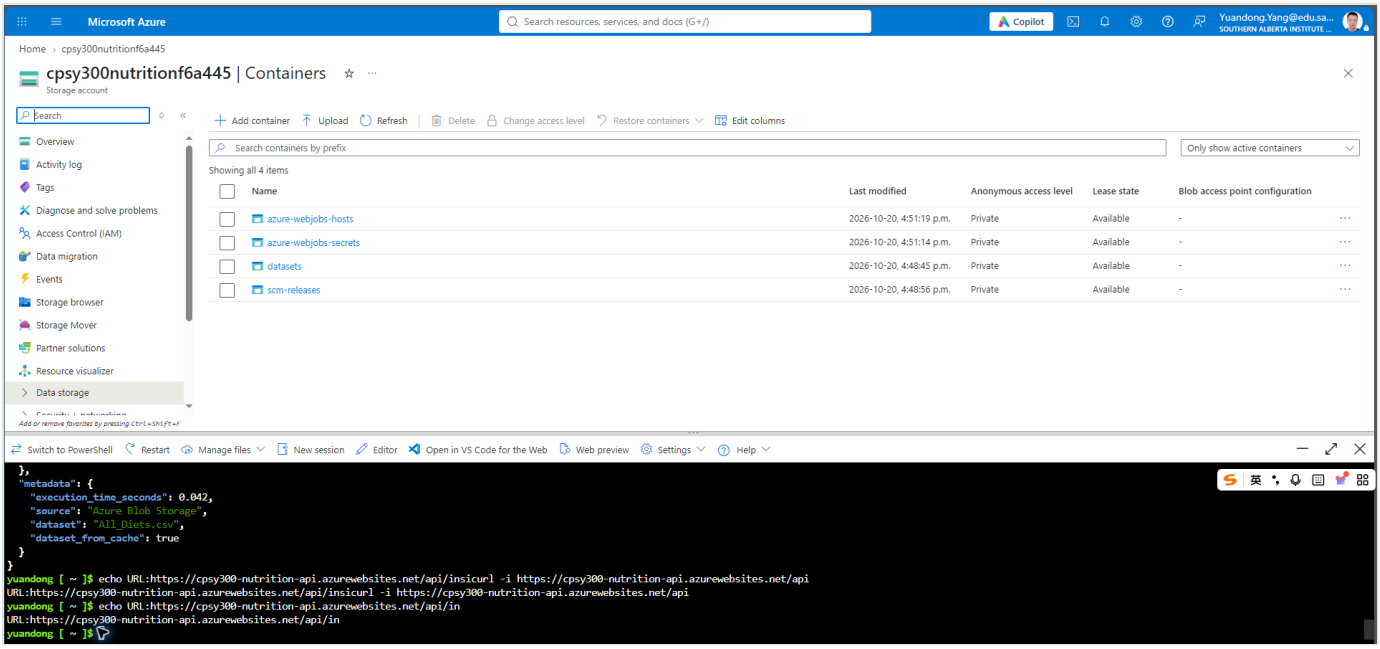
Azure portal — Function App Overview: Status Running, the `azurewebsites.net` domain, Python 3.11 and the consumption plan (`docs/screenshots/portal-function-app-essentials.png`)



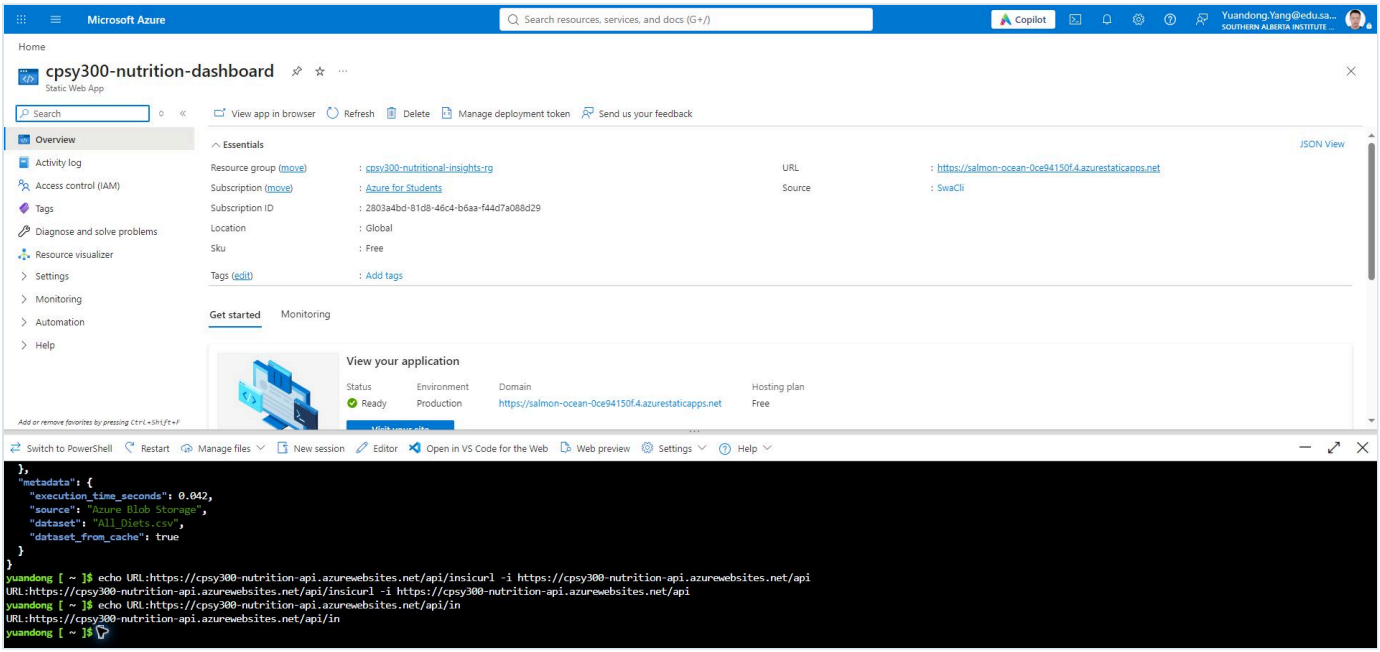
Azure portal — Function App `cpsy300-nutrition-api` with the health, insights, recipes and clusters functions (`docs/screenshots/portal-function-app.png`)



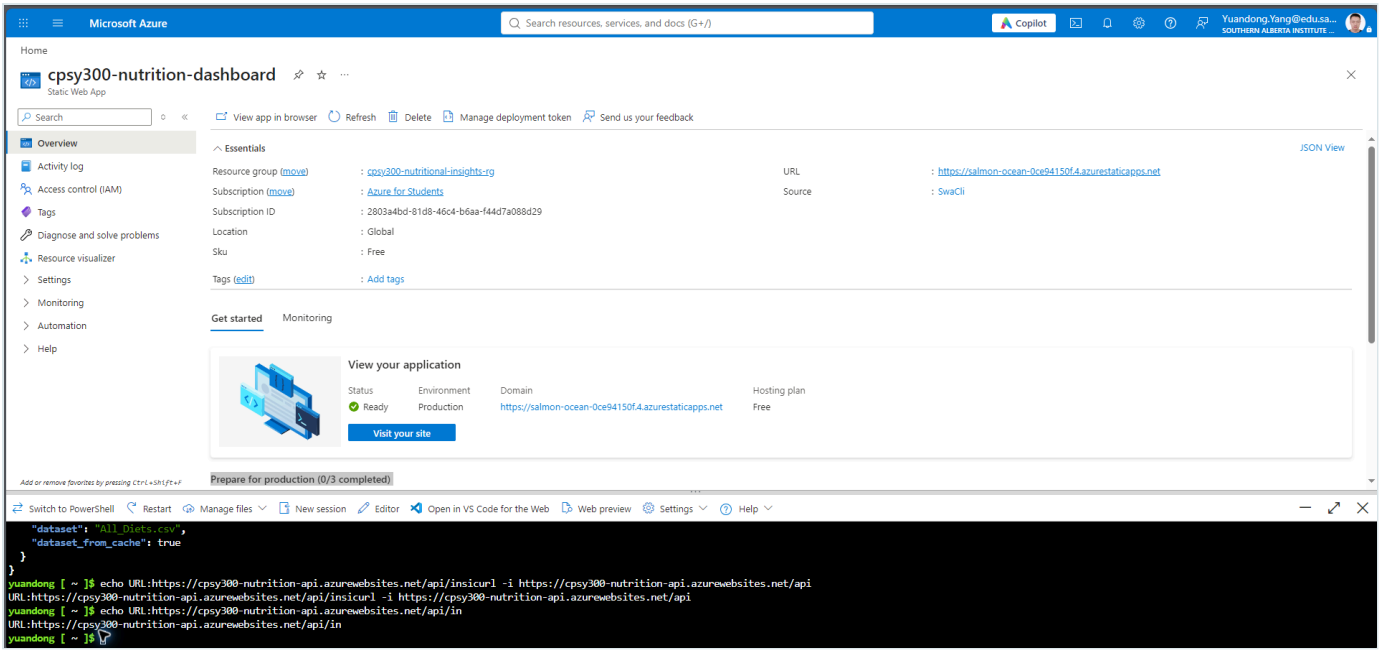
Azure portal — the `datasets` container with `All_Diets.csv` ; public access level Private (`docs/screenshots/portal-storage-container.png`)



Azure portal — the storage account Containers view with the private `datasets` container (`docs/screenshots/portal-storage-containers.png`)



Azure portal — Static Web App Overview: the default hostname, the Free SKU and region `eastus2` (`docs/screenshots/portal-static-web-app-essentials.png`)



Azure portal — Static Web App `cpsy300-nutrition-dashboard` (`docs/screenshots/portal-static-web-app.png`)

Live Site

Nutritional Insights Dashboard

CPSY 300 -- Phase 2: Cloud Dashboard Development

• 41 ms | API Connected (v2.0.0)

Filter by Diet: All Diet Types | Refresh Data

Source: Azure Blob Storage (live)

7,806	5	Keto	101.27g	Mediterranean	177,249.89g
TOTAL RECIPES	DIET TYPES	HIGHEST MEAN PROTEIN	MEAN PROTEIN (G)	HIGHEST TOTAL PROTEIN	TOTAL PROTEIN (G)

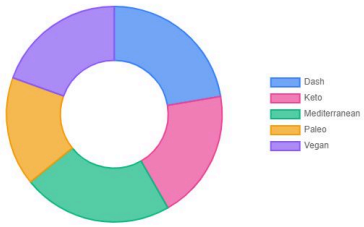
Average Macronutrients by Diet

Mean protein, carbs, and fat (grams per recipe) for each diet type.



Recipe Distribution by Diet

Number of recipes in each diet category.



Macronutrient Heatmap

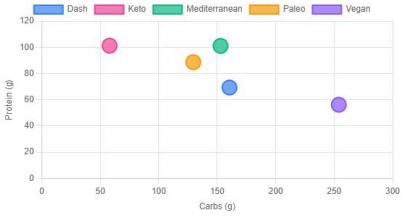
Color-coded intensity of average macronutrient values across diets.

	Dash	Keto	Mediterranean	Paleo	Vegan
Protein	69.3	101.3	101.1	88.7	56.2
Carbs	160.5	58.0	152.9	129.6	254.0
Fat	101.2	153.1	101.4	135.7	103.3

56g — 254g

Protein vs Carbs by Diet

Average protein plotted against average carbs for each diet type.



Top 5 Highest-Protein Recipes per Diet

The five recipes with the most protein in each diet category.

DIET	RECIPE	CUISINE	PROTEIN (G)	CARBS (G)
Dash	Salmon Mousse	Nordic	1239.5	22.4
Dash	Homemade Turkey Alphabet Soup	American	1190.3	49.1
Dash	Barbecue Chicken Legs	Mediterranean	1017.3	241.9
Dash	12th Man Hot Wings	American	807.0	49.2
Dash	Jazzed Up Turkey	American	517.5	575.6

Most Common Cuisines

The most frequently appearing cuisine type for each diet.

DIET TYPE	CUISINE	RECIPE COUNT
Dash	American	639
Keto	American	663
Mediterranean	Mediterranean	1,274
Paleo	American	535
Vegan	American	925

Recipe Data

Browse recipes. Use the diet filter above to narrow results.

7,806 recipes (page 1 of 157)

DIET	RECIPE NAME	CUISINE	PROTEIN (G)	CARBS (G)	FAT (G)
Paleo	Bone Broth From 'Nom Nom Paleo'	American	5.2	1.3	3.2
Paleo	Paleo Effect Asian-Glazed Pork Sides, A Sweet & Crispy Appetizer	South East Asian	181.6	28.6	146.1
Paleo	Paleo Pumpkin Pie	American	30.9	302.6	96.8
Paleo	Strawberry Guacamole recipes	Mexican	9.6	75.8	59.9
Paleo	Asian Cauliflower Fried "Rice" From 'Nom Nom Paleo'	Chinese	39.8	54.1	71.5
Paleo	Paleo Shrimp-Stuffed Mushrooms recipes	Mediterranean	68.6	34.1	42.4
Paleo	Paleo Pumpkin Pie recipes	American	30.0	275.9	97.7
Paleo	Autoimmune Paleo Pesto	Italian	4.3	14.2	72.9
Paleo	Baked Banana Chip Encrusted French Toast	French	152.9	1874.5	385.8
Paleo	Vietnamese Pho Pressure Cooker (Noodle Soup)	South East Asian	602.9	274.9	400.0

Prev | 1 | 2 | 3 | 4 | 5 | ... | 157 | Next

CPSY 300 Cloud Computing -- Project 2: Cloud Dashboard Development

Data source: All_Diets.csv (7,806 recipes across 5 diet types) via Azure Blob Storage

Live dashboard at the Static Web App URL: the green “API Connected” pill and the execution-time badge, with no offline banner — proof it renders live cloud data (`docs/screenshots/dashboard-live.png`)

Nutritional Insights Dashboard

CPSY 300 -- Phase 2: Cloud Dashboard Development

36 ms | API Connected (v2.0.0)

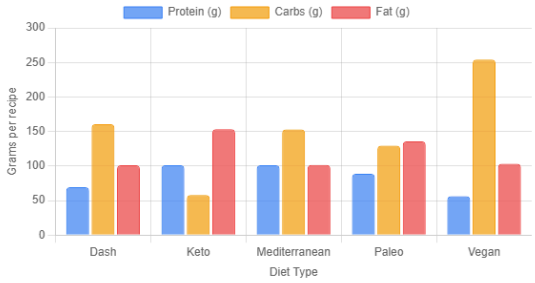
Filter by Diet: All Diet Types | Refresh Data

Source: Azure Blob Storage (live)

7,806	5	Keto	101.27g	Mediterranean	177,249.89g
TOTAL RECIPES	DIET TYPES	HIGHEST MEAN PROTEIN	MEAN PROTEIN (G)	HIGHEST TOTAL PROTEIN	TOTAL PROTEIN (G)

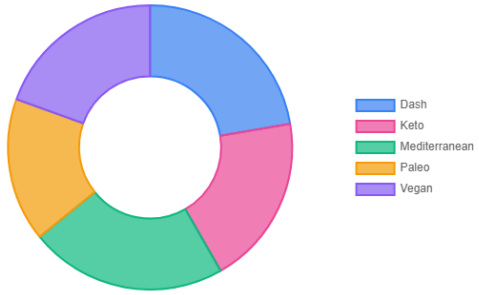
Average Macronutrients by Diet

Mean protein, carbs, and fat (grams per recipe) for each diet type.



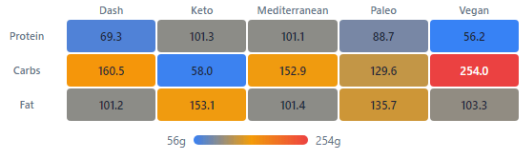
Recipe Distribution by Diet

Number of recipes in each diet category.



Macronutrient Heatmap

Color-coded intensity of average macronutrient values across diets.

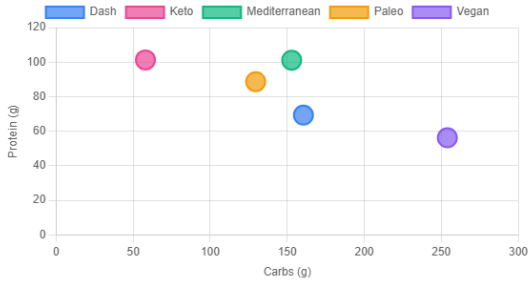


	Dash	Keto	Mediterranean	Paleo	Vegan
Protein	69.3	101.3	101.1	88.7	56.2
Carbs	160.5	58.0	152.9	129.6	254.0
Fat	101.2	153.1	101.4	135.7	103.3

56g 254g

Protein vs Carbs by Diet

Average protein plotted against average carbs for each diet type.



Top 5 Highest-Protein Recipes per Diet

The five recipes with the most protein in each diet category.

DIET	RECIPE	CUISINE	PROTEIN (G)	CARBS (G)	F (G)
Dash	Salmon Mousse	Nordic	1239.5	22.4	9
Dash	Homemade Turkey Alphabet Soup	American	1190.3	49.1	2
Dash	Barbecue Chicken Legs	Mediterranean	1017.3	241.9	1
Dash	12th Man Hot Wings	American	807.0	49.2	6
Dash	Jazzed Up Turkey	American	517.5	575.6	

Most Common Cuisines

The most frequently appearing cuisine type for each diet.

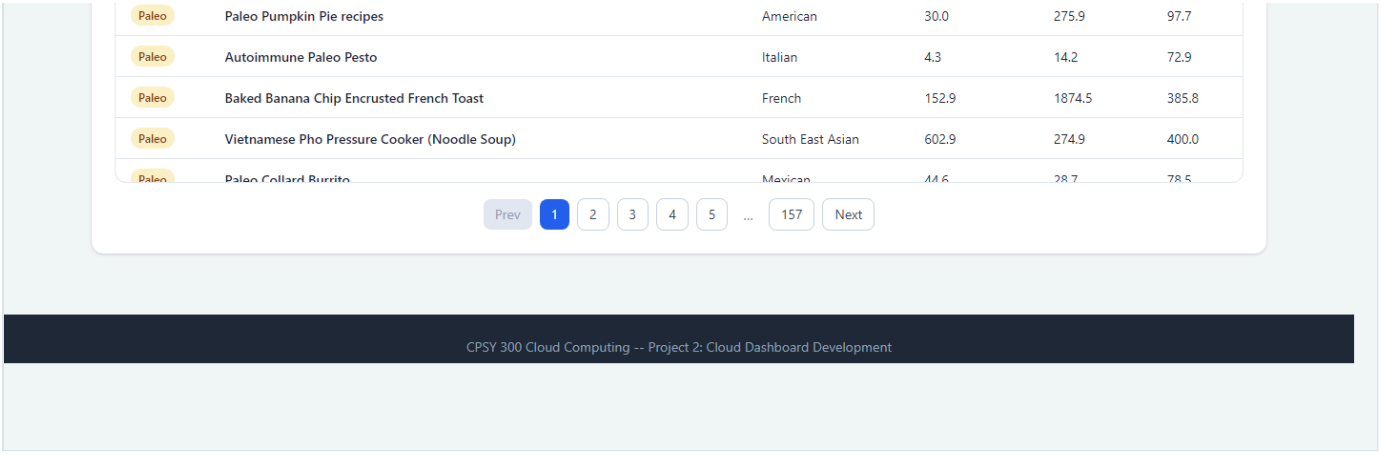
DIET TYPE	CUISINE	RECIPE COUNT
Dash	American	639
Keto	American	663
Mediterranean	Mediterranean	1,274
Paleo	American	535
Vegan	American	925

Recipe Data

Browse recipes. Use the diet filter above to narrow results.

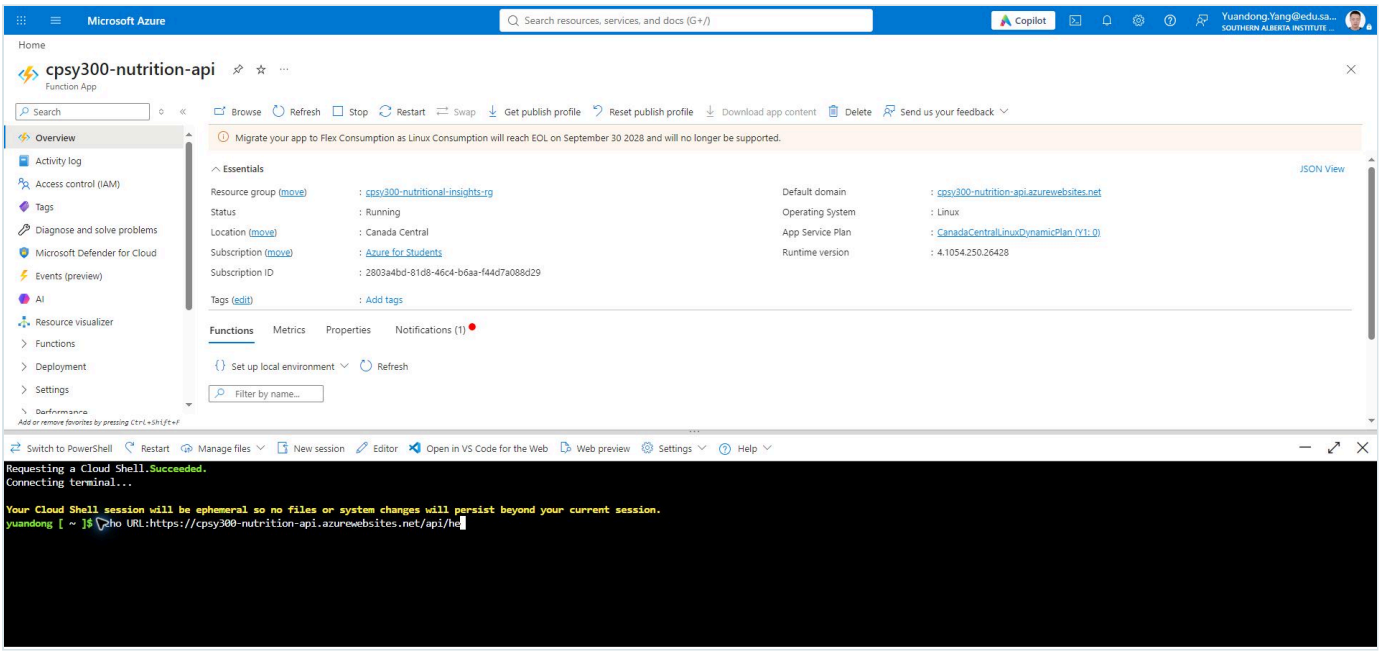
7,806 recipes (page 1 of 157)

DIET	RECIPE NAME	CUISINE	PROTEIN (G)	CARBS (G)	FAT (G)
Paleo	Bone Broth From 'Nom Nom Paleo'	American	5.2	1.3	3.2
Paleo	Paleo Effect Asian-Glazed Pork Sides, A Sweet & Crispy Appetizer	South East Asian	181.6	28.6	146.1
Paleo	Paleo Pumpkin Pie	American	30.9	302.6	96.8
Paleo	Strawberry Guacamole recipes	Mexican	9.6	75.8	59.9
Paleo	Asian Cauliflower Fried "Rice" From 'Nom Nom Paleo'	Chinese	39.8	54.1	71.5
Paleo	Paleo Shrimp-Stuffed Mushrooms recipes	Mediterranean	68.6	34.1	42.4

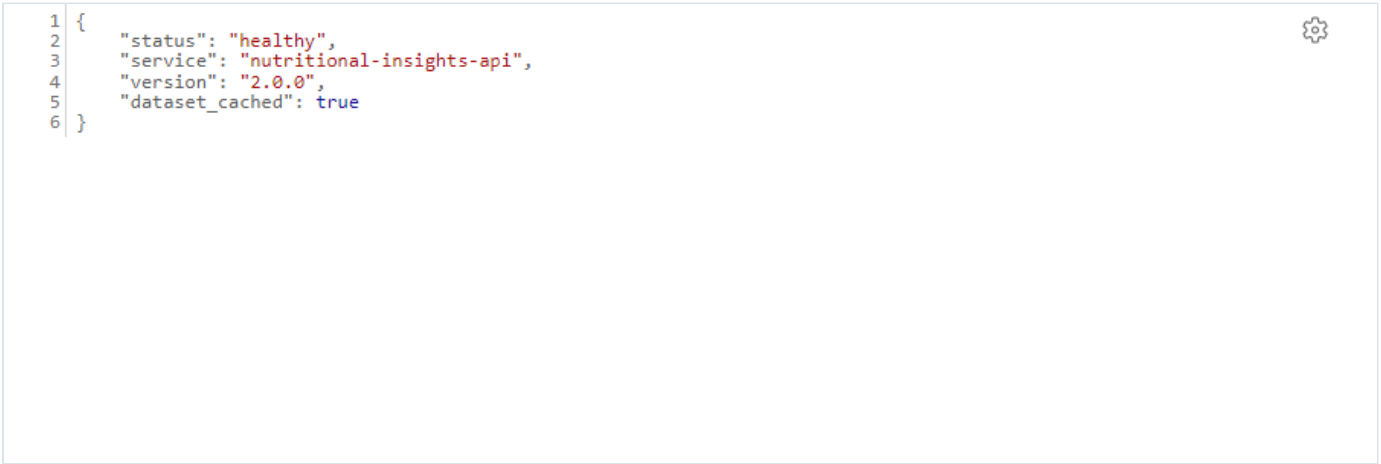


The full live dashboard: four charts, the stat tiles and the paginated recipe table (`docs/screenshots/dashboard-full-page.png`)

API Responses



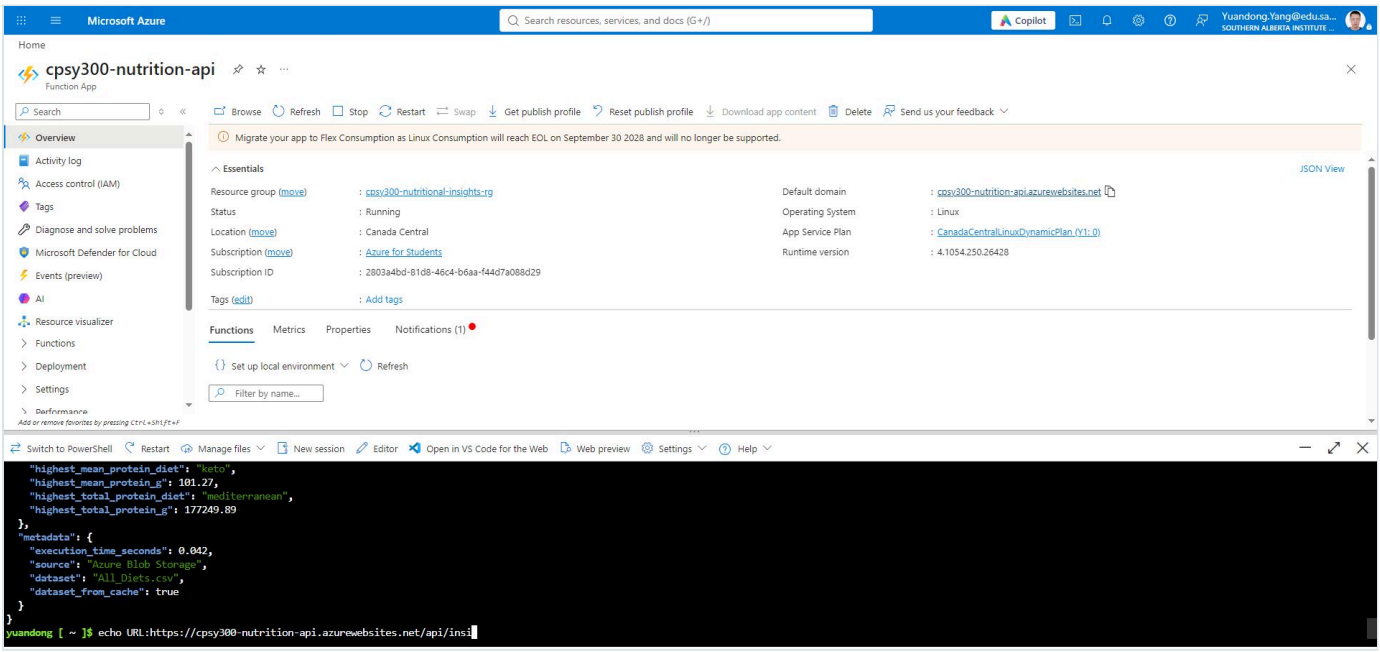
GET `/api/health` on the live Function App: status healthy, 200 OK (`docs/screenshots/api-health-browser.png`)



GET `/api/health` from the command line: the JSON body and the 200 status (`docs/screenshots/api-health-curl.png`)

```
{
    "average_macros": [
        {
            "Diet_type": "dash",
            "Protein(g)": 69.28,
            "Carbs(g)": 160.54,
            "Fat(g)": 101.15
        },
        {
            "Diet_type": "keto",
            "Protein(g)": 101.27,
            "Carbs(g)": 57.97,
            "Fat(g)": 153.12
        },
        {
            "Diet_type": "mediterranean",
            "Protein(g)": 101.11,
            "Carbs(g)": 152.91,
            "Fat(g)": 101.42
        },
        {
            "Diet_type": "paleo",
            "Protein(g)": 88.67,
            "Carbs(g)": 129.55,
            "Fat(g)": 135.67
        },
        {
            "Diet_type": "vegan",
            "Protein(g)": 56.16,
            "Carbs(g)": 254,
            "Fat(g)": 103.3
        }
    ],
    "top_recipes": [
        {
            "Diet_type": "dash",
            "Recipe_name": "Salmon Mousse",
            "Cuisine_type": "nordic",
            "Protein(g)": 1239.47,
            "Carbs(g)": 22.4,
            "Fat(g)": 983.36
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "Homemade Turkey Alphabet Soup",
            "Cuisine_type": "american",
            "Protein(g)": 1190.35,
            "Carbs(g)": 49.13,
            "Fat(g)": 297.45
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "Barbecue Chicken Legs",
            "Cuisine_type": "mediterranean",
            "Protein(g)": 1017.25,
            "Carbs(g)": 241.93,
            "Fat(g)": 1002.07
        },
        {
            "Diet_type": "dash",
            "Recipe_name": "12th Man Hot Wings",
```


GET `/api/insights` in the browser: average macros, correlation and the summary block (`docs/screenshots/api-insights-browser.png`)



GET `/api/insights` response: `summary.total_recipes` 7,806 and the `metadata` block with `execution_time_seconds` (`docs/screenshots/api-insights-response.png`)

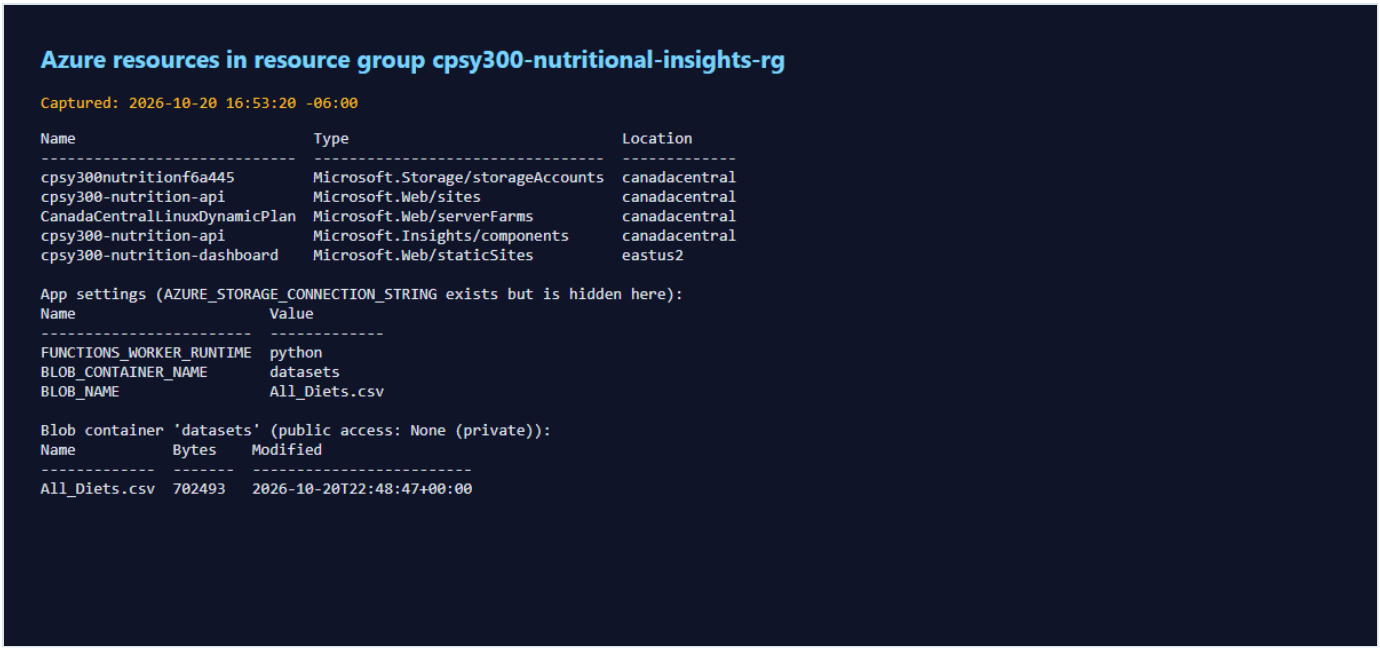
```
{
    "recipes": [
        {
            "Diet_type": "paleo",
            "Recipe_name": "Bone Broth From 'Nom Nom Paleo'",
            "Cuisine_type": "american",
            "Protein(g)": 5.22,
            "Carbs(g)": 1.29,
            "Fat(g)": 3.2
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Paleo Effect Asian-Glazed Pork Sides, A Sweet & Crispy Appetizer",
            "Cuisine_type": "south east asian",
            "Protein(g)": 181.55,
            "Carbs(g)": 28.62,
            "Fat(g)": 146.14
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Paleo Pumpkin Pie",
            "Cuisine_type": "american",
            "Protein(g)": 30.91,
            "Carbs(g)": 302.59,
            "Fat(g)": 96.76
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Strawberry Guacamole recipes",
            "Cuisine_type": "mexican",
            "Protein(g)": 9.62,
            "Carbs(g)": 75.78,
            "Fat(g)": 59.89
        },
        {
            "Diet_type": "paleo",
            "Recipe_name": "Asian Cauliflower Fried \"Rice\" From 'Nom Nom Paleo'",
            "Cuisine_type": "chinese",
            "Protein(g)": 39.84,
            "Carbs(g)": 54.08,
            "Fat(g)": 71.55
        }
    ],
    "count": 5,
    "total": 7806,
    "page": 1,
```


GET /api/recipes?diet_type=keto&page=2&limit=50 : count , total , page , limit and total_pages (docs/screenshots/api-recipes-response.png)

CLI & Verification



CORS scoped to the dashboard origin and the measured response timing (docs/screenshots/cli-cors-and-timing.png)



Azure CLI resource inventory for the resource group — every deployed resource in one list (`docs/screenshots/cli-resource-inventory.png`)

---

07 / Challenges and Solutions

Challenge	Solution
Duplicate CORS headers Host-level CORS *and* handler-written CORS produced the `Access-Control-Allow-Origin` header twice, which browsers reject.	Give exactly one layer ownership: the host allows only the SWA origin and the function code emits no CORS headers at all.
A 1.21 MB response on every load `/api/recipes` returned all 7,806 rows on every load and filter change — slow and costly on a consumption plan.	Page server-side (default 50, hard cap 500); return `total` / `total_pages` ; a fresh request per page click. Typical response is now tens of KB.
Re-downloading and re-cleaning the CSV each request Doing it per request dominated response time, but caching blindly would ignore a re-uploaded CSV.	Cache the cleaned DataFrame keyed by the blob **ETag**; one cheap metadata call per request; `dataset_from_cache` exposes the path taken.
The API URL is unknown until deploy time The Function URL only exists after the Function App is created, and hard-coding it breaks other environments.	`index.html` ships a `__FUNCTION_API_URL__` placeholder; `deploy.sh` substitutes it into a throwaway `build/` , verifies it, and deploys only that. Sources stay clean.

Challenge	Solution
A silent fallback that looked live An earlier build rendered non-live data behind a subtle badge — actively misleading.	Fallback is now the real Project 1 output behind a prominent banner; any failure switches the **whole** view, so live and fallback data never mix.
Static Web Apps region availability `az staticwebapp create` fails in `canadacentral`, where everything else lives.	A separate `SWA_LOCATION=eastus2` for the Static Web App while `LOCATION=canadacentral` governs the rest; both overridable and documented.
A Static Web App without a GitHub token Linking a SWA to GitHub needs an over-privileged personal access token.	Create the SWA **unlinked** and deploy with the `swa` CLI using a deployment token from `az staticwebapp secrets list` — no GitHub credential.
Keeping the docs and code in sync An earlier draft and the deployed code had drifted apart on endpoint counts, correlation signs and resource names.	Reconciled both against a verified audit: corrected the counts and signs, aligned every resource name with `deploy.sh`, and added the `/api/clusters` route to the cloud backend so the live API matches the Stage 1 contract and the dashboard's clusters chart.

08 / Teardown and Cost

```
cd stage3
./teardown.sh                            # deletes cpsy300-nutritional-insights-rg and everything in it
az group list --query "[].name" -o tsv   # confirm the group is gone
```

> **Run teardown as soon as the evidence is captured.** Azure bills by the hour and by the request, so a resource group left running over a weekend costs more than the whole demo did. The delete is irreversible: the uploaded dataset, the Function App settings and the Static Web App all go with it. Re-running `deploy.sh` rebuilds everything, but the Static Web App gets a **new hostname**, so any URL already written into a report or a slide stops working.

Sources: `division.md`, `stage3/docs/phase2-documentation.md`, `deploy.sh` / `teardown.sh`, and the Azure deployment evidence captured on 2026-10-20. Live API: https://cpsy300-nutrition-api.azurewebsites.net/api · Dashboard: https://salmon-ocean-0ce94150f.4.azurestaticapps.net/